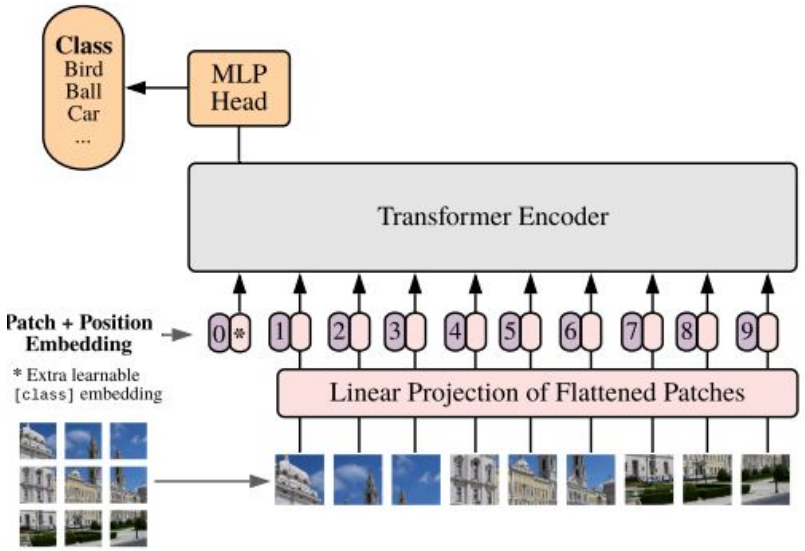
Computer Vision

2023-24 First Semester, M.Tech (AIML)



Session #-9:

Attention, Transformers & Vision Transformers (ViT)

S. P. Vimal | *CSIS Department* | *WILP* | *BITS – Pilani* (*vimalsp@wilp.bits-pilani.ac.in*)

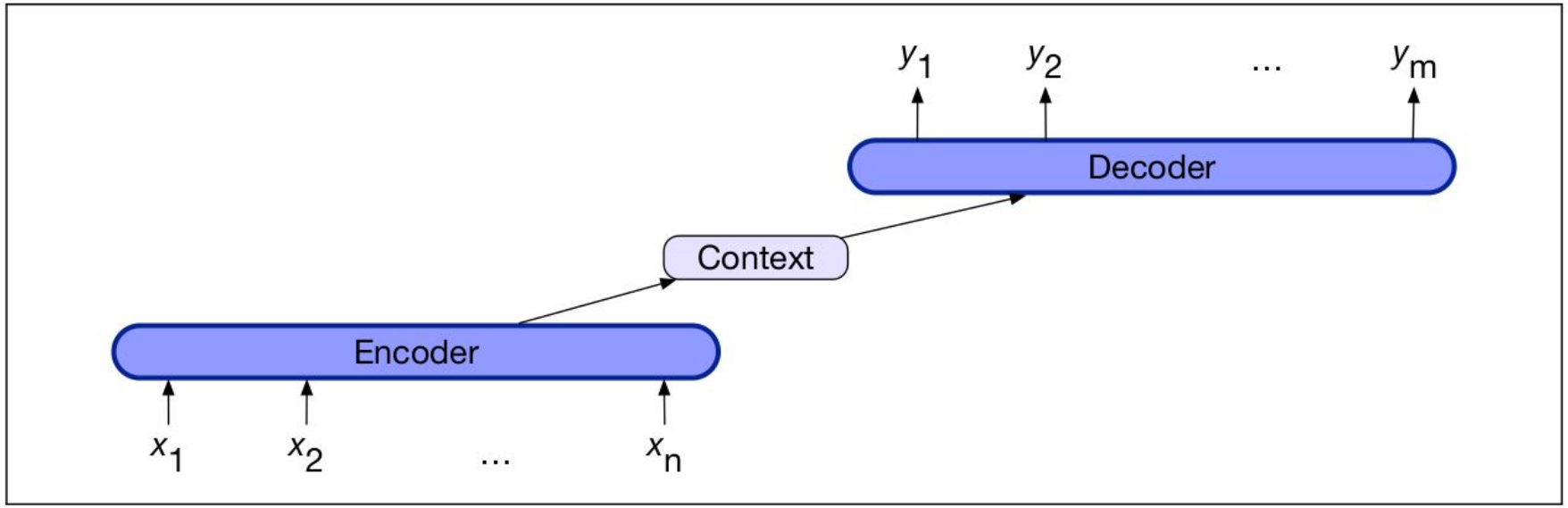
BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Topics

- ➔ Attention (Review)
- ➔ Transformers (Review)
- ➔ Vision Transformer (ViT)
- ➔ Applications of ViT in Computer Vision
- ➔ Popular ViT Variants

Acknowledgement: Slide Materials adopted from - Intro to Computer Vision (Cornell Tech); Noah Snavely

(Review) Encoder - Decoder Arch for Seq. to Seq. Translation

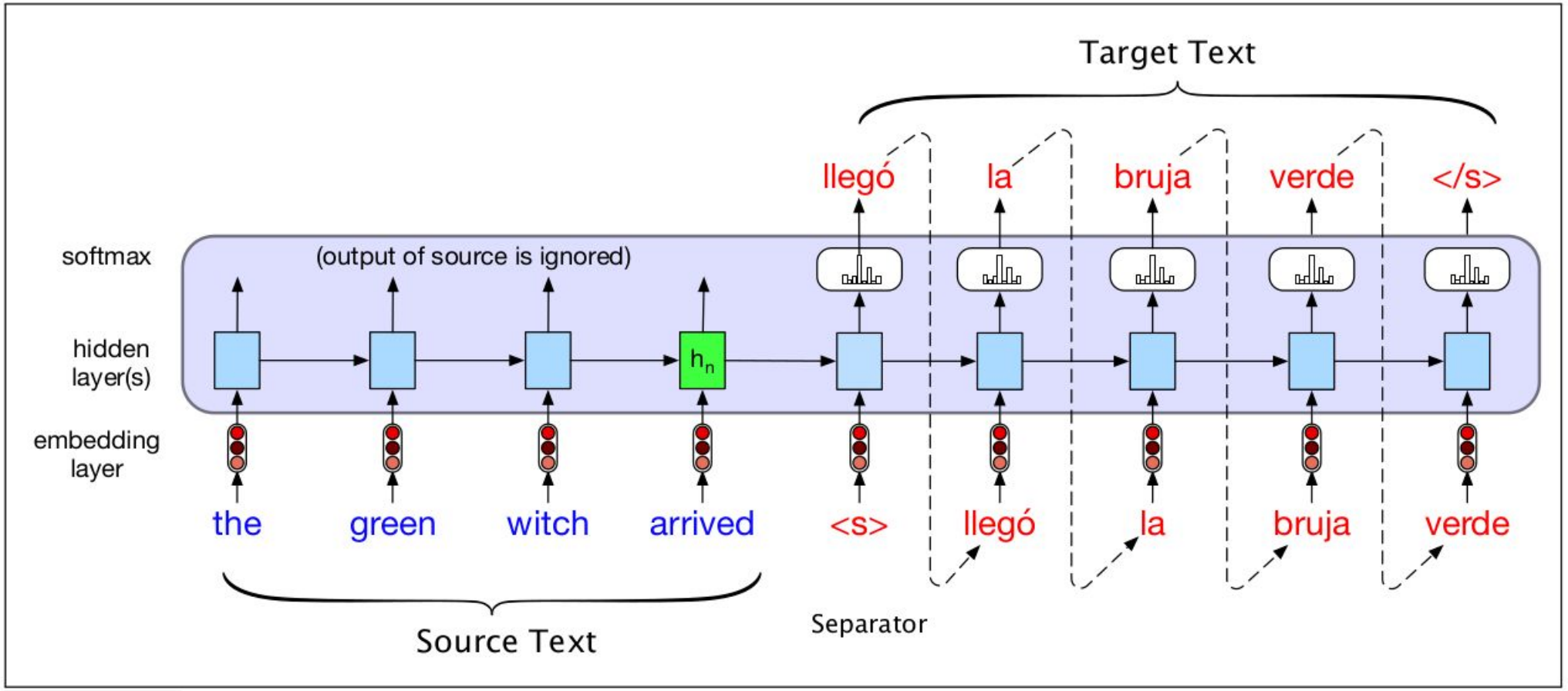


Components:

1. Encoder
2. Context Vector, c
3. Decoder

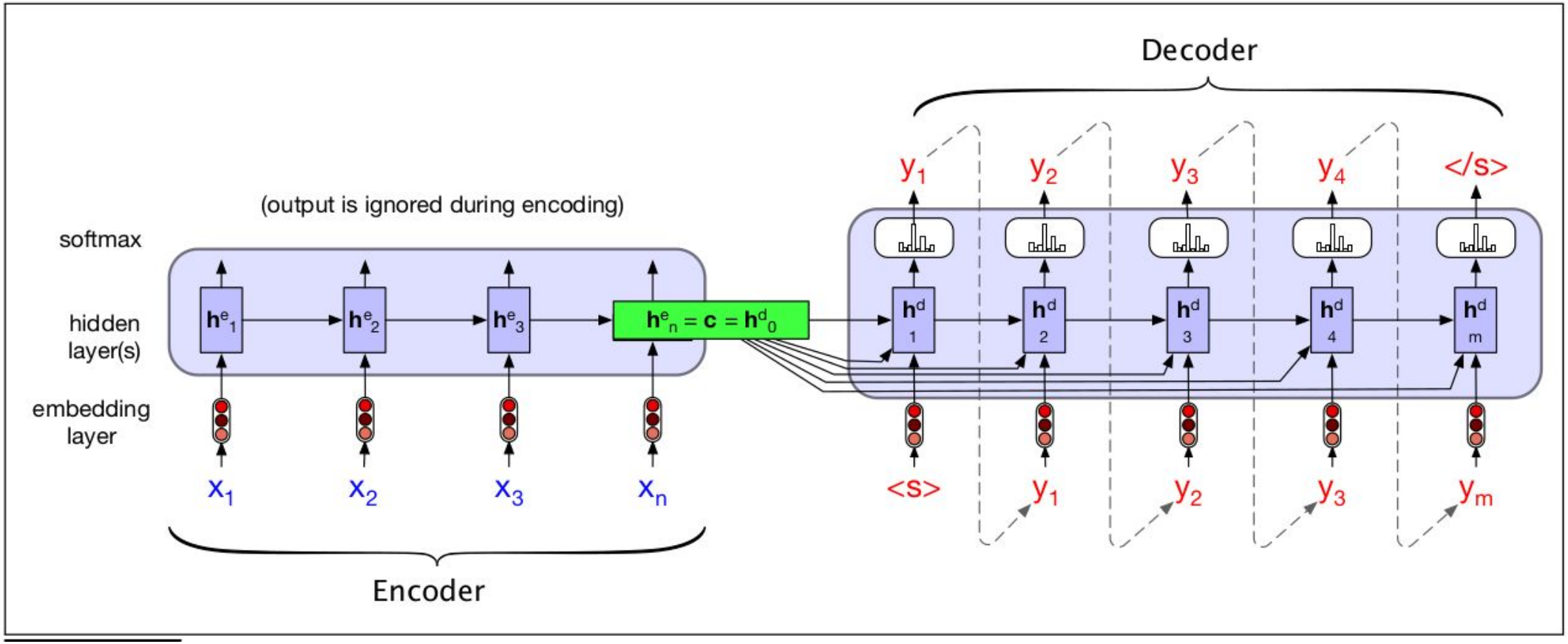
Note: The context is a function of the hidden representations of the input, and may be used by the decoder in a variety of ways

(Review) Encoder - Decoder Arch for Seq. to Seq. Translation



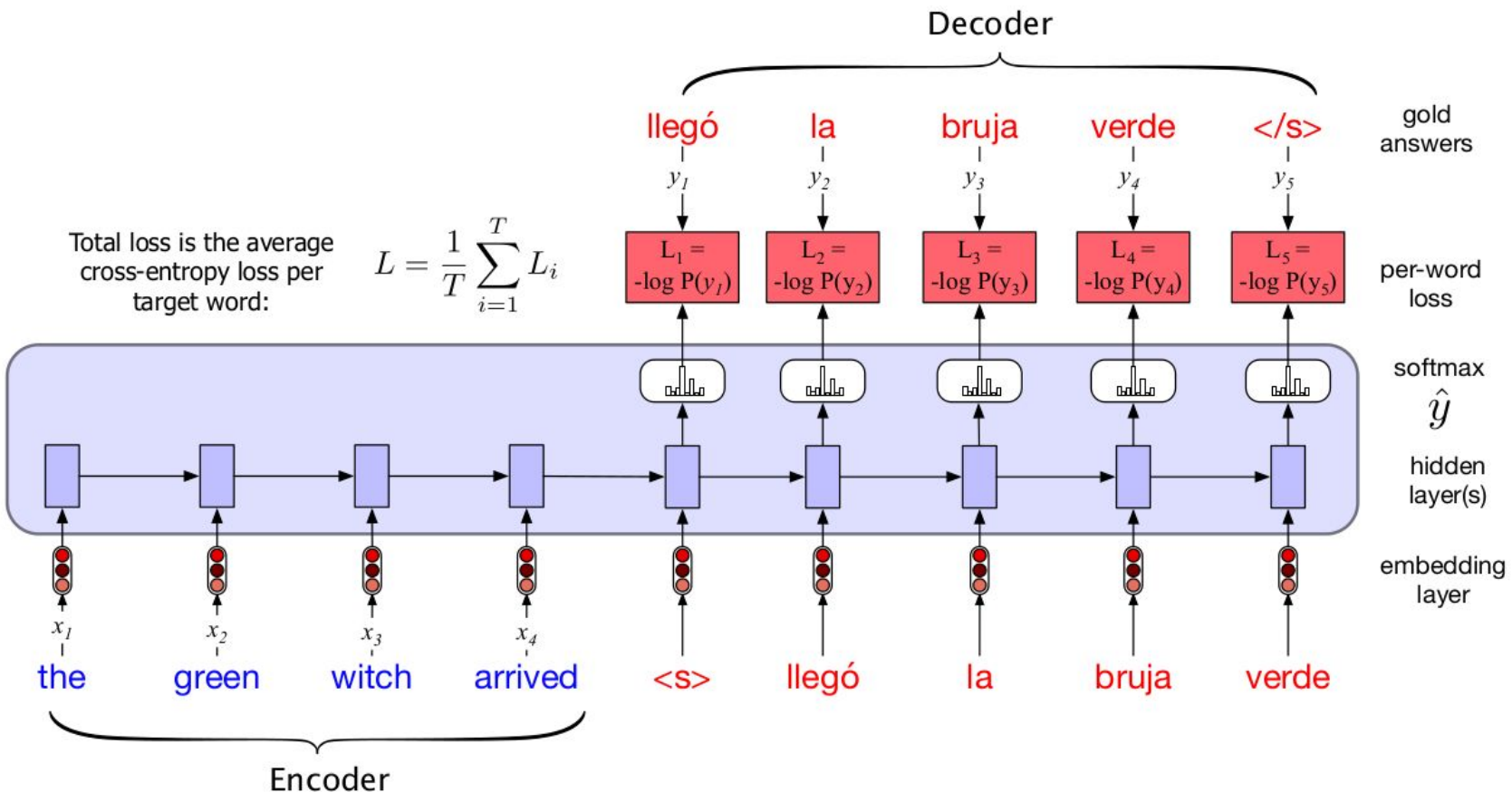
We make the context vector c available to more than just the first decoder hidden state, to ensure that the influence of the context vector, c, doesn't wane as the output sequence is generated.

(Review) Encoder - Decoder Arch for Seq. to Seq. Translation



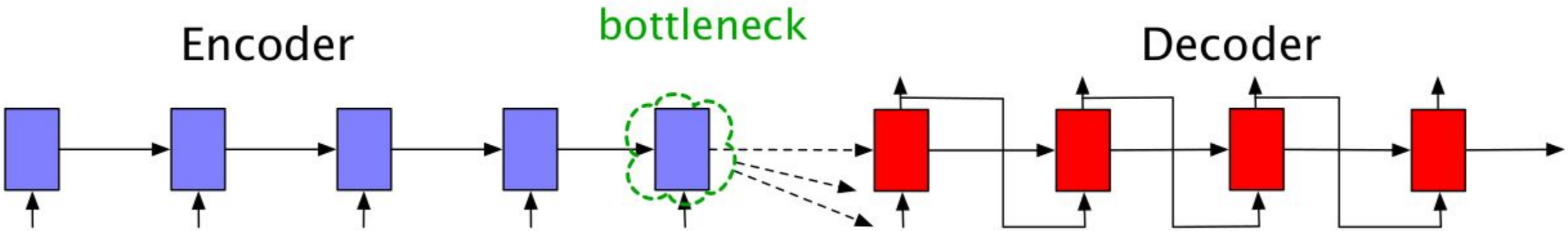
Source and target sentences are concatenated with a separator token in between, and the decoder uses context information from the encoder's last hidden state.

(Review) Encoder - Decoder Arch for Seq. to Seq. Translation



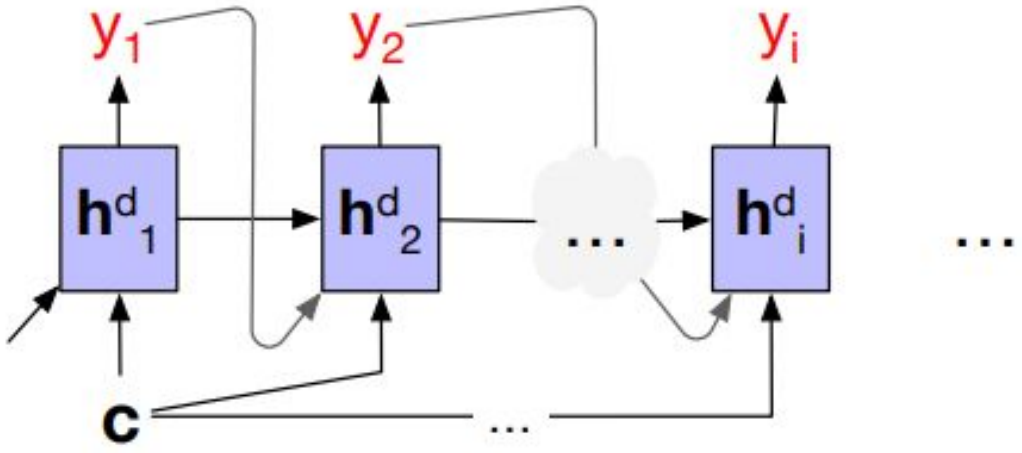
In the decoder we usually don't propagate the model's softmax outputs ŷt , but use teacher forcing. We compute the softmax output distribution over ŷ in the decoder, which can then be averaged to compute a loss for the sentence.

(Review) Encoder - Decoder Arch for Seq. to Seq. Translation



Requiring the context c to be only the encoder's final hidden state forces all the information from the entire source sentence to pass through this representational bottleneck.

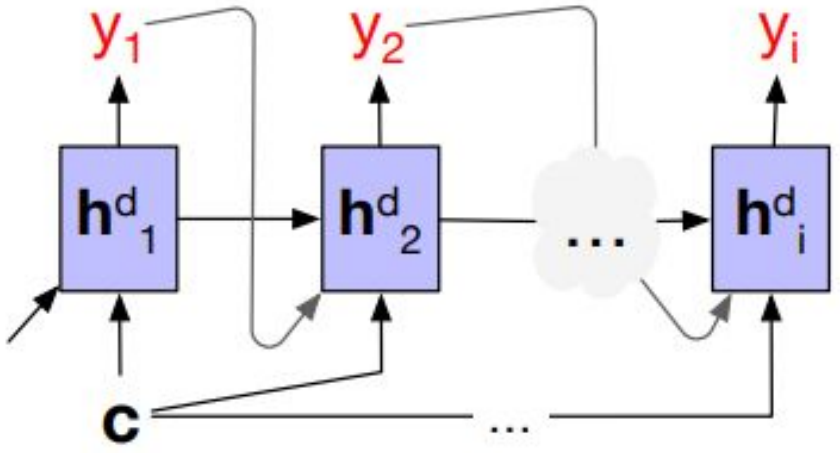
(Review) Attention !

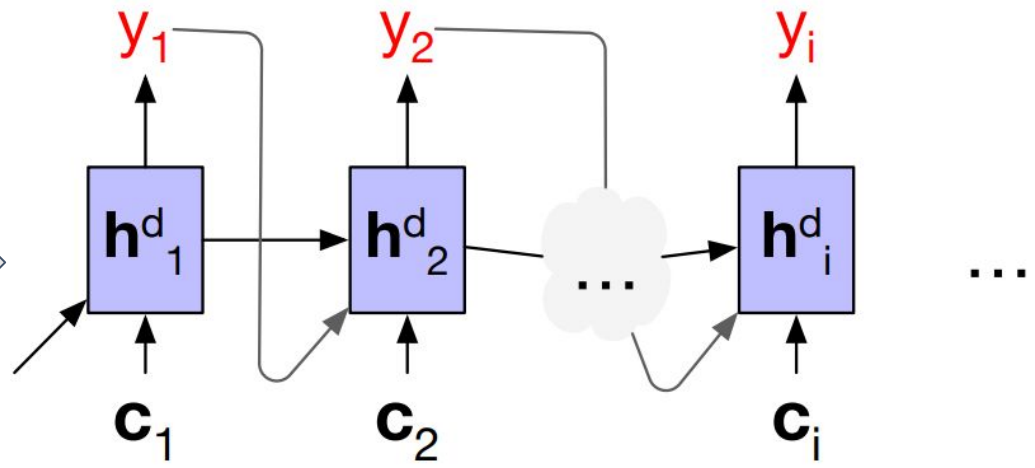


Without attention, a decoder sees the same context vector , which is a static function of all the encoder hidden states

$$\mathbf{h}_t^d = g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c})$$

(Review) Attention !





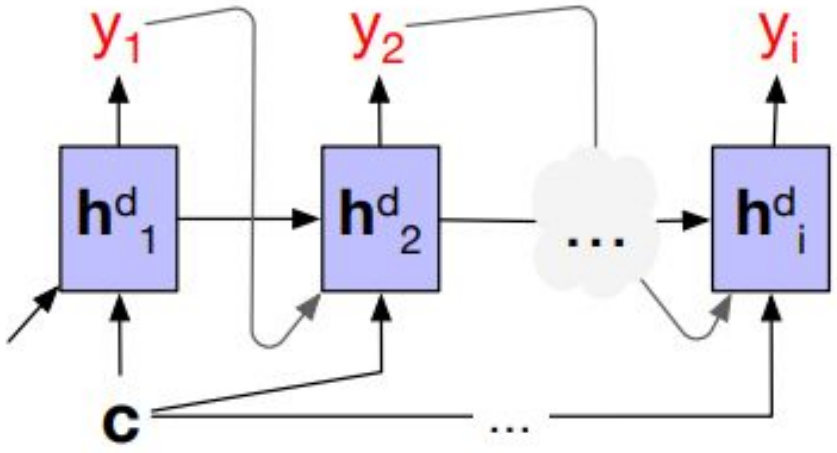
Without attention, a decoder sees the same context vector , which is a static function of all the encoder hidden states

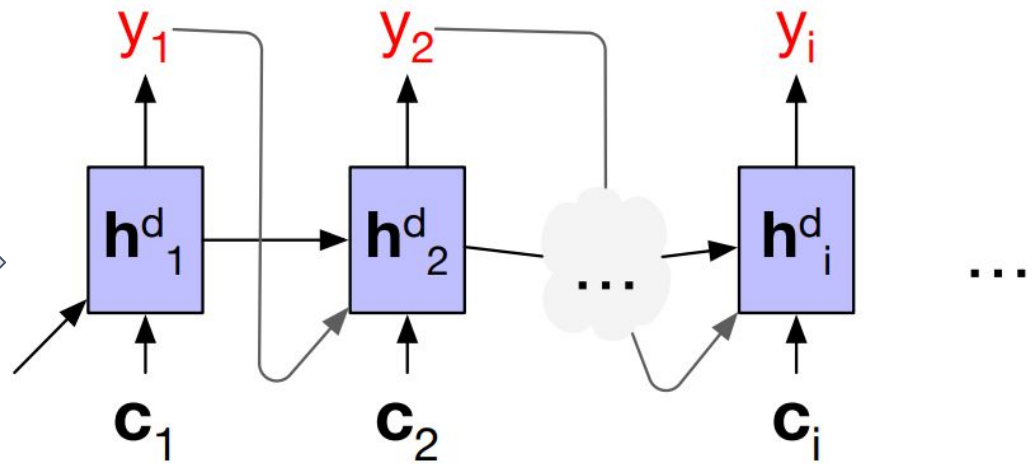
With attention, decoder to sees a different, dynamic, context, which is a function of all the encoder hidden states

$$\mathbf{h}_t^d = g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c})$$

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$

(Review) Attention !





Without attention, a decoder sees the same context vector , which is a static function of all the encoder hidden states

With attention, decoder to sees a different, dynamic, context, which is a function of all the encoder hidden states

$$\mathbf{h}_t^d = g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c})$$

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$

With attention, decoder gets information from all the hidden states of the encoder, not just the last hidden state of the encoder Each context vector is obtained by taking a weighted sum of all the encoder hidden states.

The weights focus on ('attend to') a particular part of the source text that is relevant for the token the decoder is currently producing

(Review) Attention !

Step -1 : Find out how relevant each encoder state is to the present decoder state $\mathbf{h}_{i-1}^{d}$

Compute a score of similarity between $\mathbf{h}_{i-1}^{d}$ and all the encoder states : $score(\mathbf{h}_{i-1}^{d}, \mathbf{h}_{j}^{e})$

Dot Product Attention : $\text{score}(\mathbf{h}_{i-1}^{d}, \mathbf{h}_{j}^{e}) = \mathbf{h}_{i-1}^{d} \cdot \mathbf{h}_{j}^{e}$

Step -2 : Normalize all the scores with softmax to create a vector of weights, $\alpha_{i,j}$

$\alpha_{i,j}$ indicates the proportional relevance of each encoder hidden state j to the prior hidden decoder state, $\mathbf{h}_{i-1}^{d}$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{h}_{i-1}^{d}, \mathbf{h}_{j}^{e}) \ \forall j \in e)$$

$$= \frac{\exp(\text{score}(\mathbf{h}_{i-1}^{d}, \mathbf{h}_{j}^{e})}{\sum_{k} \exp(\text{score}(\mathbf{h}_{i-1}^{d}, \mathbf{h}_{k}^{e}))}$$

(Review) Attention !

Step -3 : Given the distribution in α, compute a fixed-length context vector for the current decoder state by taking a weighted average over all the encoder hidden states

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$

(Review) Attention !

Step -3 : Given the distribution in α, compute a fixed-length context vector for the current decoder state by taking a weighted average over all the encoder hidden states

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$

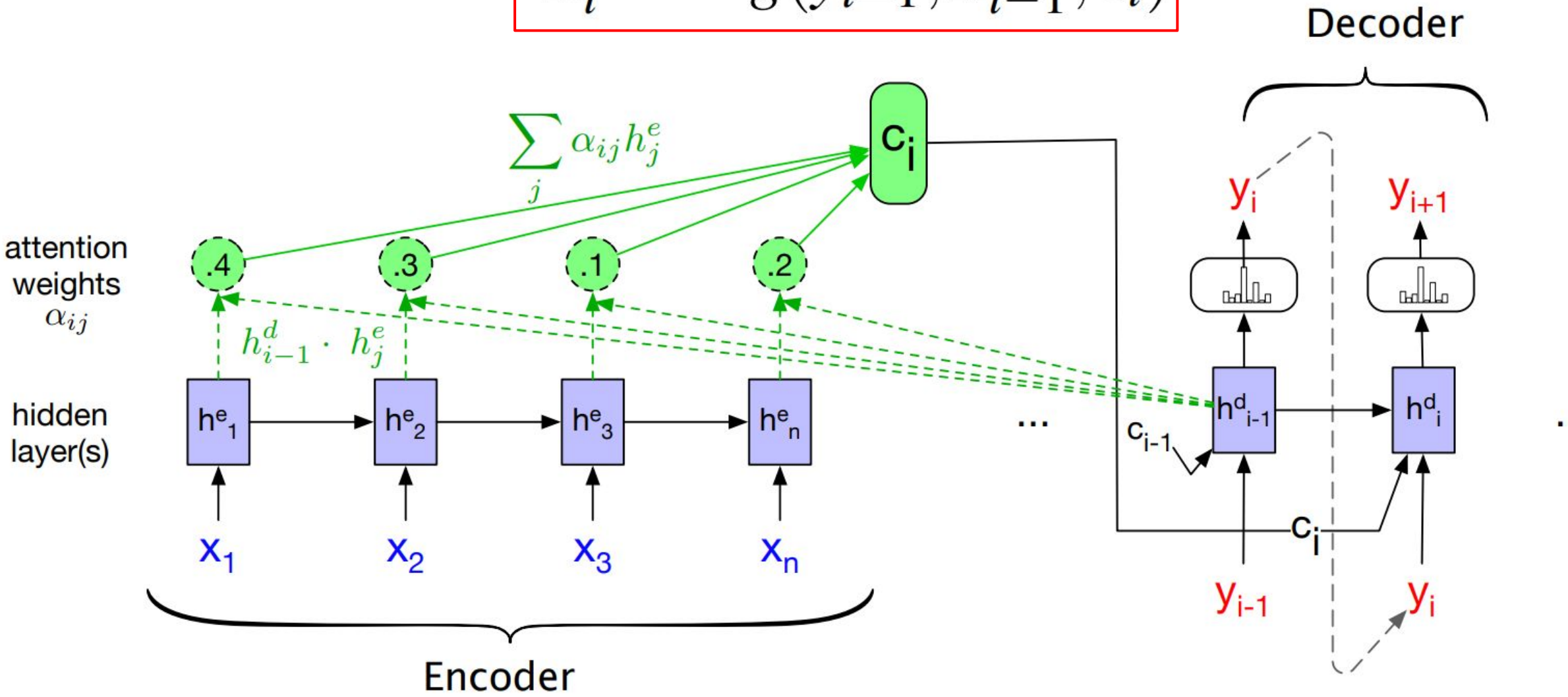
Plus : In step-1, we can get a more powerful scoring function by parameterizing the score with its own set of weights, W_s:

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{t-1}^d \mathbf{W}_s \mathbf{h}_j^e$$

W_s , is trained during normal end-to-end training,
W_s , gives the network the ability to learn which aspects of similarity between the decoder and encoder states are important to the current application.

(Review) Attention !

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$



(Review) Transformers

- 2017, NIPS, Vaswani et. al., Attention Is All You Need !!!
- Transformers map sequences of input vectors $(x_1,\ldots,x_n)$ to sequences of output vectors $(y_1,\ldots,y_n)$ of the same length
- Made up of transformer blocks in which the key component is self-attention layers

  [Self-attention allows a network to directly extract and use information from arbitrarily large contexts directly !!!]

- Transformers are not based on recurrent connections ⇒ Parallel implementations possible ⇒ Efficient to scale (comparing LSTM)

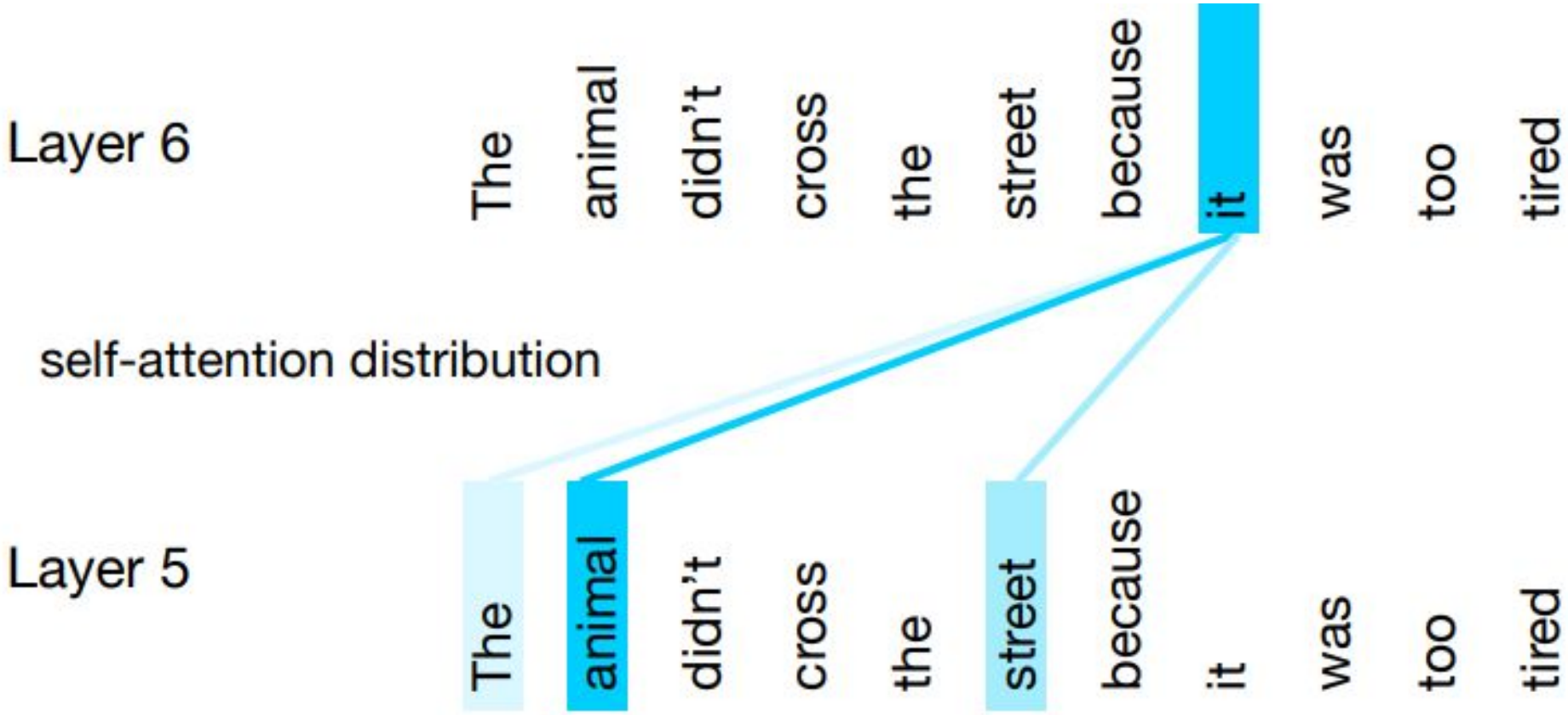
(Review) Self-Attention | Transformers

- Attention ⇒ Ability to compare an item of interest to a collection of other items in a way that reveals their relevance in the current context.
- Self-attention ⇒

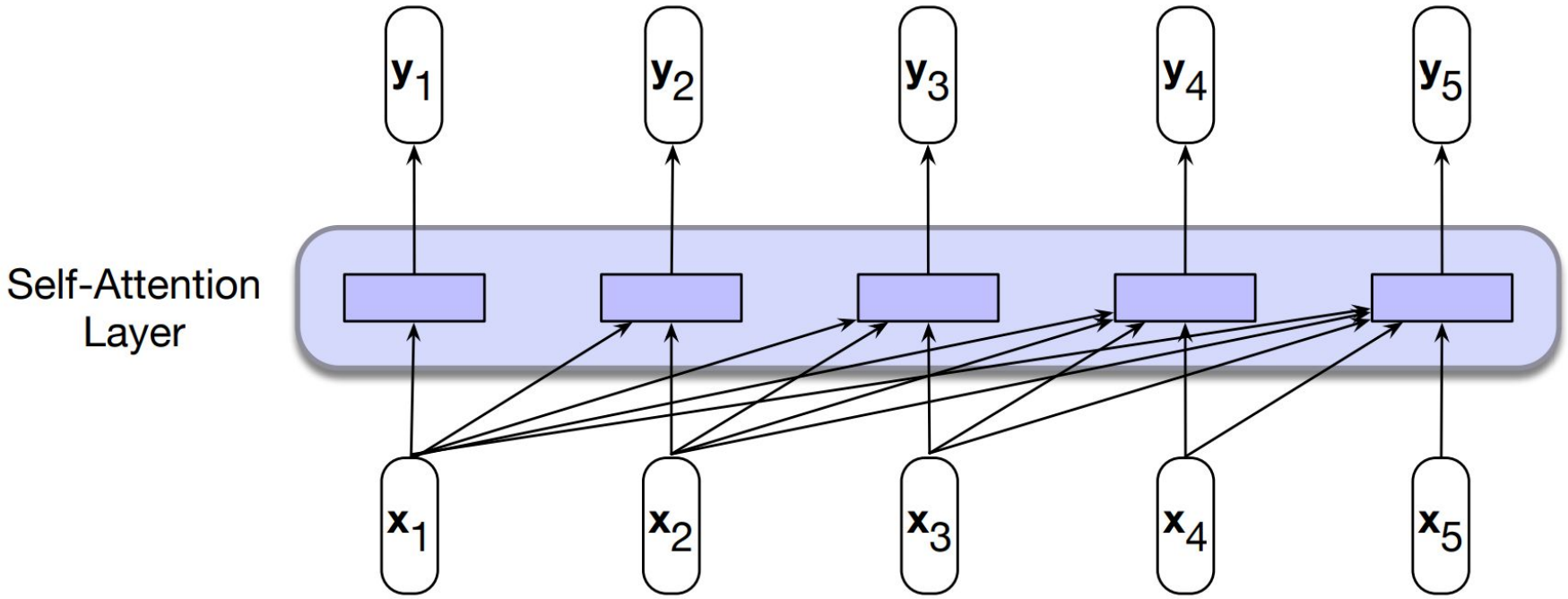
  > Set of *comparisons* are to other elements *within a given sequence*

  > Use these comparisons to compute an output for the current input

(Review) Self-Attention | Transformers



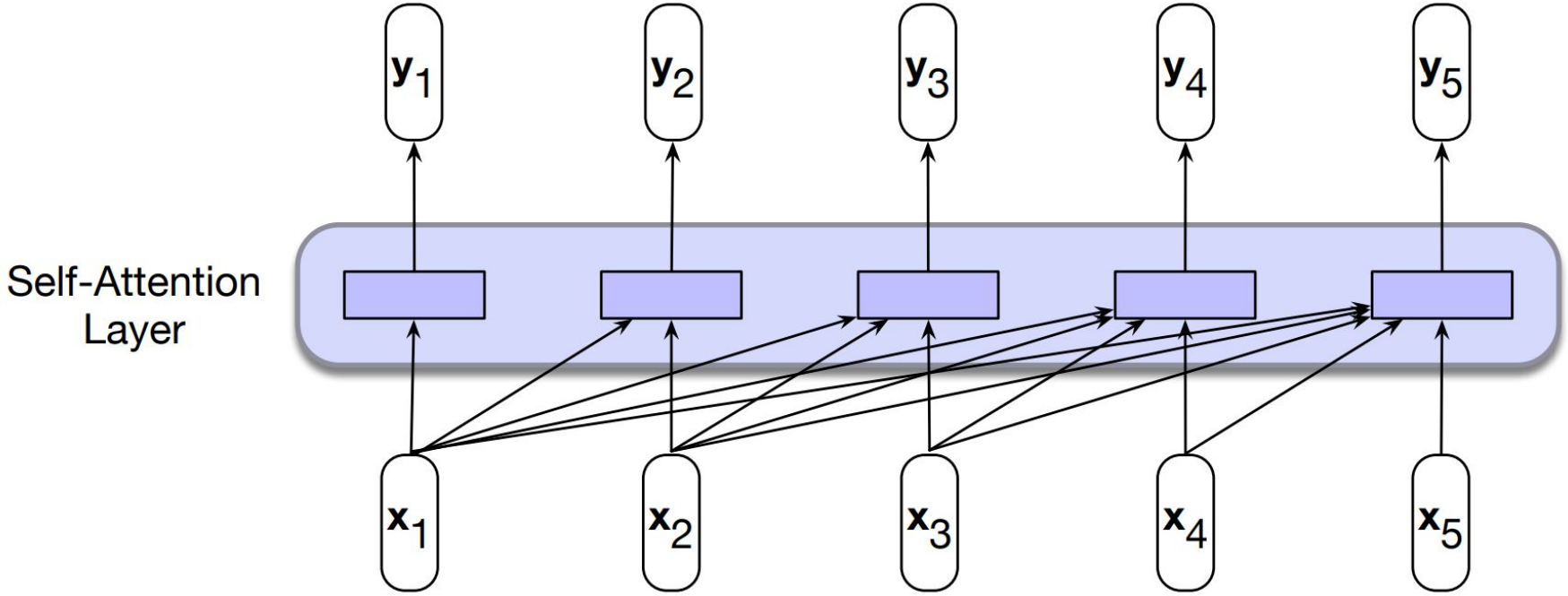
(Review) Self-Attention | Transformers



In processing each element of the sequence, the model attends to all the inputs up to, and including, the current one.

Unlike RNNs, the computations at each time step are independent of all the other steps and therefore can be performed in parallel.

(Review) Self-Attention | Transformers



$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\mathbf{y}_i = \sum_{j \le i} \alpha_{ij} \mathbf{x}_j$$

$$\begin{aligned} \alpha_{ij} &= \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \quad \forall j \le i \\ &= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_{k=1}^{i} \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \quad \forall j \le i \end{aligned}$$

(Review) Self-Attention | Transformers

- Let us understand how transformers uses self-attention !

(Review) Self-Attention | Transformers

$$\mathbf{W}^V, \mathbf{W}^K, \mathbf{W}^Q \in \mathbb{R}^{d \times d}$$

In Vaswani et al., 2017, d was 1024.

- Let us understand how transformers uses self-attention !

$$\mathbf{q}_i = \mathbf{W}^{\mathbf{Q}}\mathbf{x}_i \qquad \mathbf{k}_i = \mathbf{W}^{\mathbf{K}}\mathbf{x}_i \qquad \mathbf{v}_i = \mathbf{W}^{\mathbf{V}}\mathbf{x}_i$$

Query, Q	Key, K	Value, V
As the current focus of attention when being compared to all of the other preceding inputs.	In its role as a preceding input being compared to the current focus of attention.	As a value used to compute the output for the current focus of attention

Three different roles each x_i (input embedding) , in the computation of self attention

(Review) Self-Attention | Transformers

$$\mathbf{W}^V, \mathbf{W}^K, \mathbf{W}^Q \in \mathbb{R}^{d \times d}$$

In Vaswani et al., 2017, d was 1024.

- Let us understand how transformers uses self-attention !

$$\mathbf{q}_i = \mathbf{W}^{\mathbf{Q}}\mathbf{x}_i \qquad \mathbf{k}_i = \mathbf{W}^{\mathbf{K}}\mathbf{x}_i \qquad \mathbf{v}_i = \mathbf{W}^{\mathbf{V}}\mathbf{x}_i$$

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

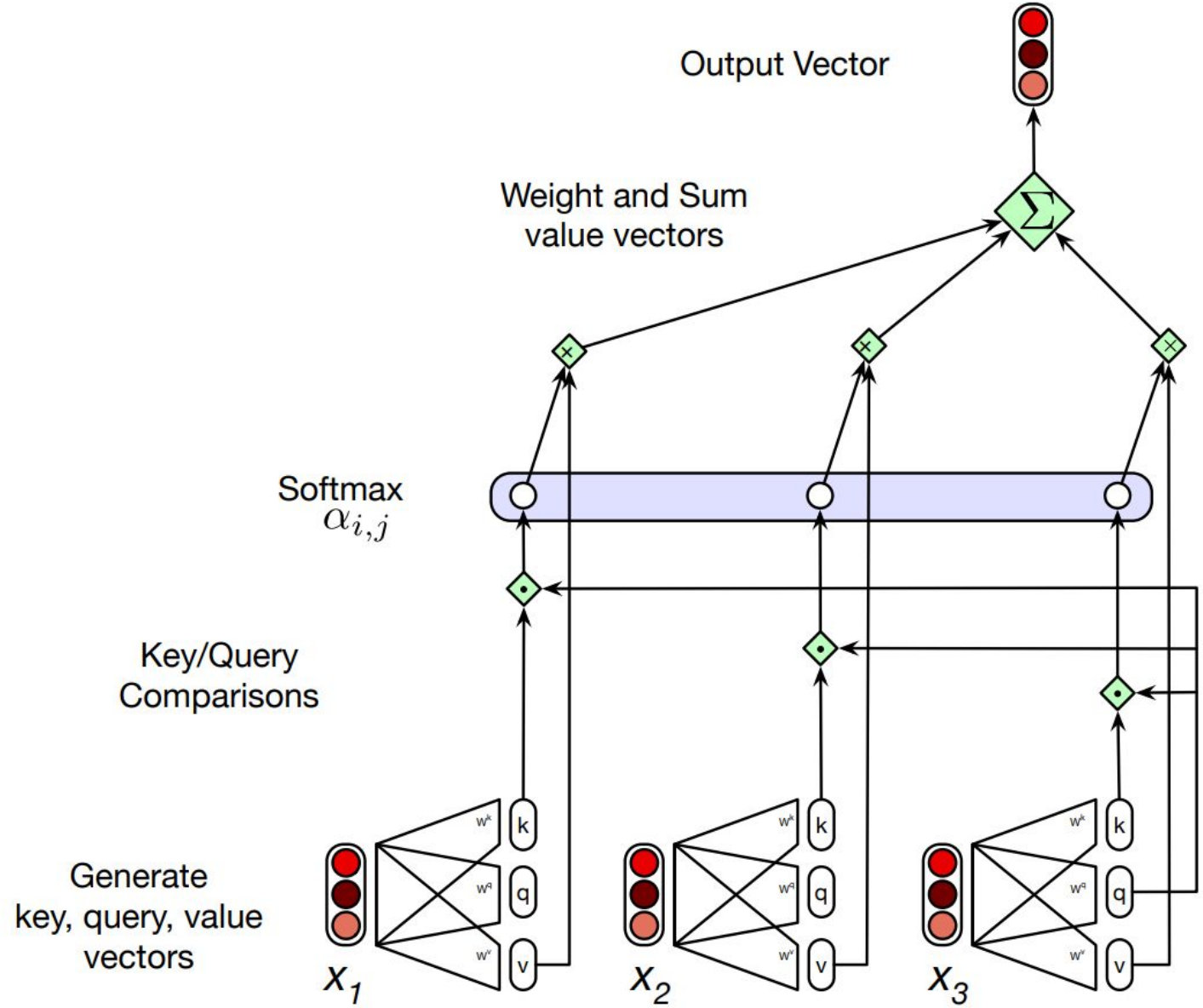
The simple dot product can be an arbitrarily large;
scaled dot-product is used in transformers;

$$\begin{aligned} \alpha_{ij} &= \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \ \forall j \leq i \\ &= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_{k=1}^{i} \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \ \forall j \leq i \end{aligned}$$

$$\mathbf{y}_i = \sum_{j \leq i} \alpha_{ij} \mathbf{v}_j$$

(Review) Self-Attention | Transformers

- Each output, y_i , is computed independently
- Entire process can be parallelized



Calculating the value of y3, the third element of a sequence using causal (left-to-right) self-attention

(Review) Self-Attention | Transformers

- Pack the input embeddings of the N input tokens into a single matrix

$$\mathbf{X} \in \mathbb{R}^{N \times d}$$

> Each row of X is the embedding of one token of the input

- Multiply X by the key, query, and value (dxd) matrices

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{\mathbf{Q}}; \;\; \mathbf{K} = \mathbf{X}\mathbf{W}^{\mathbf{K}}; \;\; \mathbf{V} = \mathbf{X}\mathbf{W}^{\mathbf{V}}$$

(Review) Self-Attention | Transformers

$$\text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d_k}}\right)\mathbf{V}$$

$\mathbf{QK}^\intercal$ Matrix

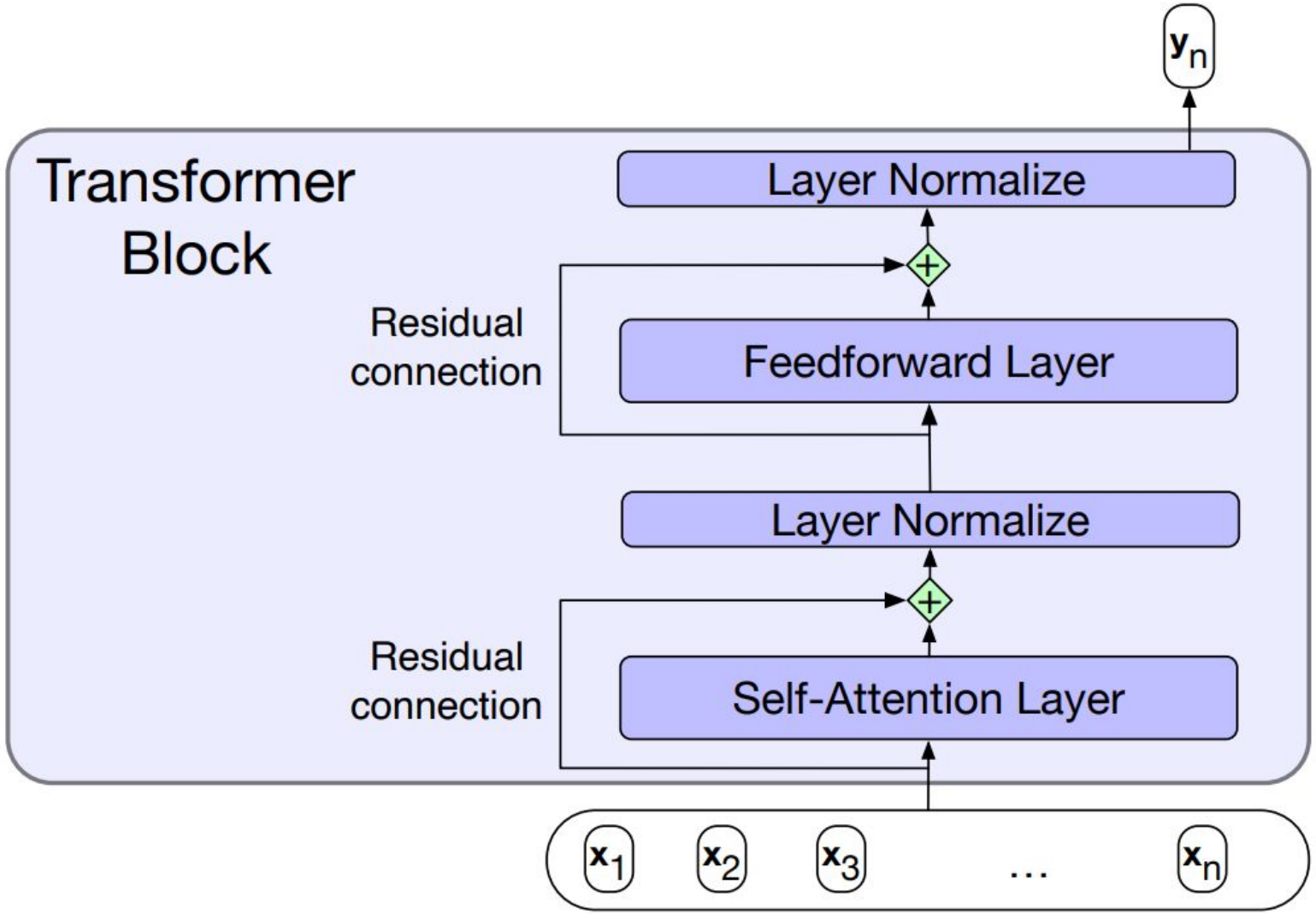
N

q1•k1	−∞	−∞	−∞	−∞
q2•k1	q2•k2	−∞	−∞	−∞
q3•k1	q3•k2	q3•k3	−∞	−∞
q4•k1	q4•k2	q4•k3	q4•k4	−∞
q5•k1	q5•k2	q5•k3	q5•k4	q5•k5

N

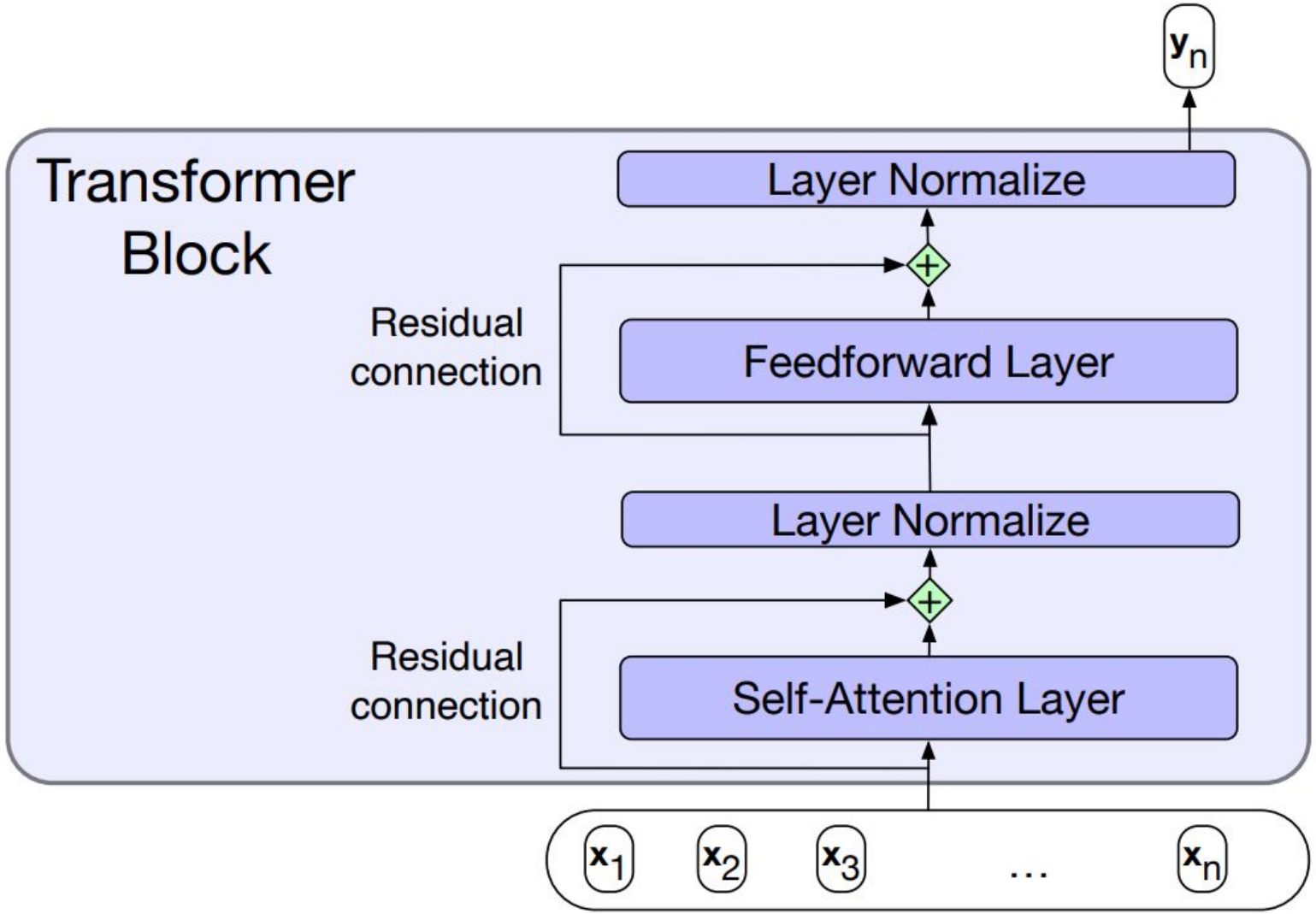
Note: Upper-triangle portion of the comparisons matrix zeroed out (set to −∞, which the softmax will turn to zero)

(Review) Transformer Blocks | Transformers



$$\mathbf{z} = \text{LayerNorm}(\mathbf{x} + \text{SelfAttention}(\mathbf{x}))$$
$$\mathbf{y} = \text{LayerNorm}(\mathbf{z} + \text{FFN}(\mathbf{z}))$$

(Review) Transformer Blocks | Transformers



$$\mathbf{z} = \text{LayerNorm}(\mathbf{x} + \text{SelfAttention}(\mathbf{x}))$$
$$\mathbf{y} = \text{LayerNorm}(\mathbf{z} + \text{FFN}(\mathbf{z}))$$

LayerNorm:

$$\mu = \frac{1}{d_h}\sum_{i=1}^{d_h} x_i$$

$$\sigma = \sqrt{\frac{1}{d_h}\sum_{i=1}^{d_h}(x_i - \mu)^2}$$

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

$$\text{LayerNorm} = \gamma\,\hat{\mathbf{x}} + \beta$$

(Review) Multihead-Attention | Transformers

- Different words in a sentence can relate to each other in many different ways simultaneously

  >> A single transformer block to learn to capture all of the different kinds of parallel relations among its inputs is inadequate.

- Multihead self-attention layers

  >> Heads ⇒ sets of self-attention layers, that reside in parallel layers at the same depth in a model, each with its own set of parameters.

  >> Each head learn different aspects of the relationships that exist among inputs at the same level of abstraction

(Review) Multihead-Attention | Transformers

- Different words in a sentence can relate to each other in many different ways simultaneously

  >> A single transformer block to learn to capture all of the different kinds of parallel relations among its inputs is inadequate.

- Multihead self-attention layers

  >> Heads ⇒ sets of self-attention layers, that reside in parallel layers at the same depth in a model, each with its own set of parameters.

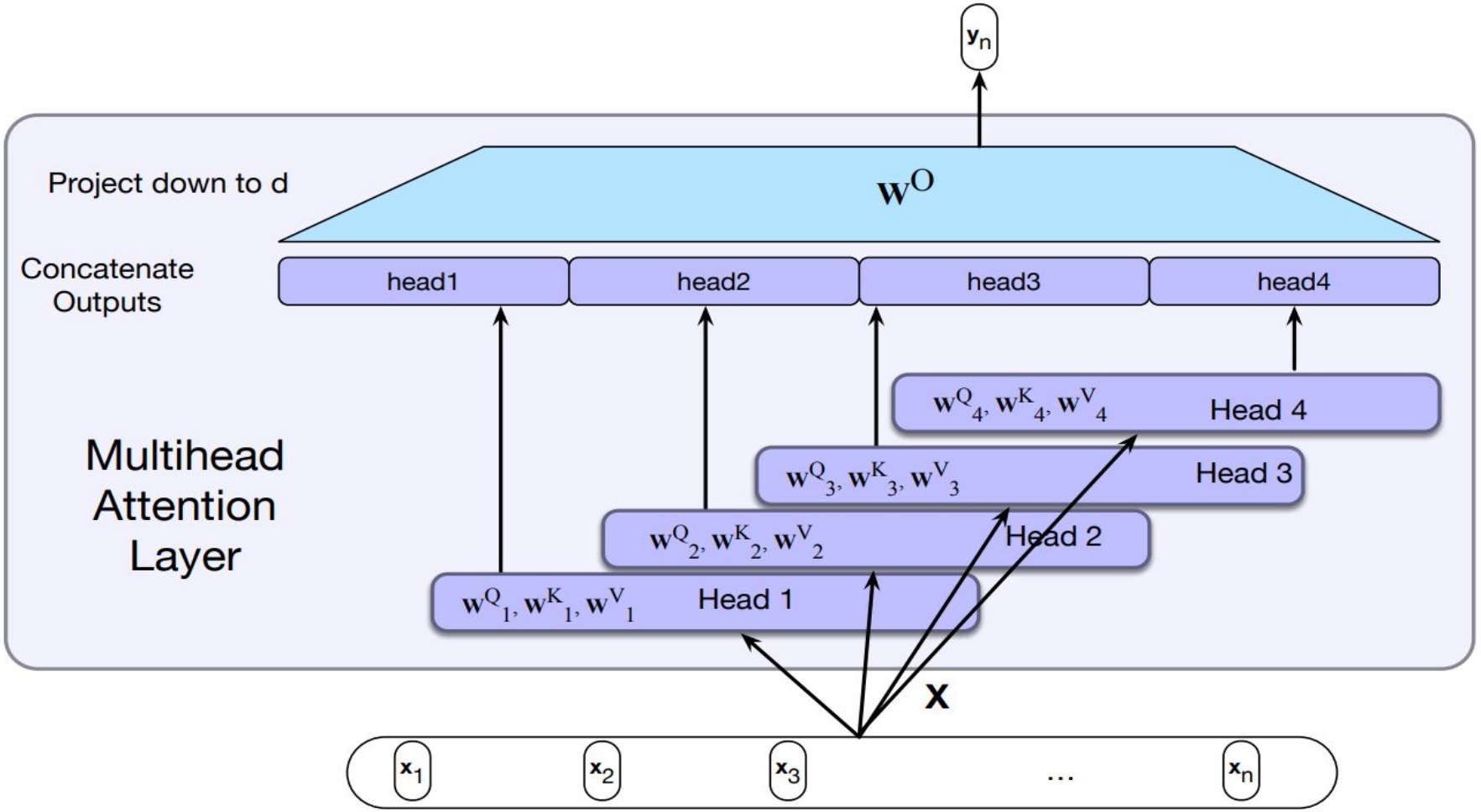
  >> Each head learn different aspects of the relationships that exist among inputs at the same level of abstraction

$$\text{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \ldots \oplus \mathbf{head}_h)\mathbf{W}^O$$

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_i^Q \; ; \; \mathbf{K} = \mathbf{X}\mathbf{W}_i^K \; ; \; \mathbf{V} = \mathbf{X}\mathbf{W}_i^V$$

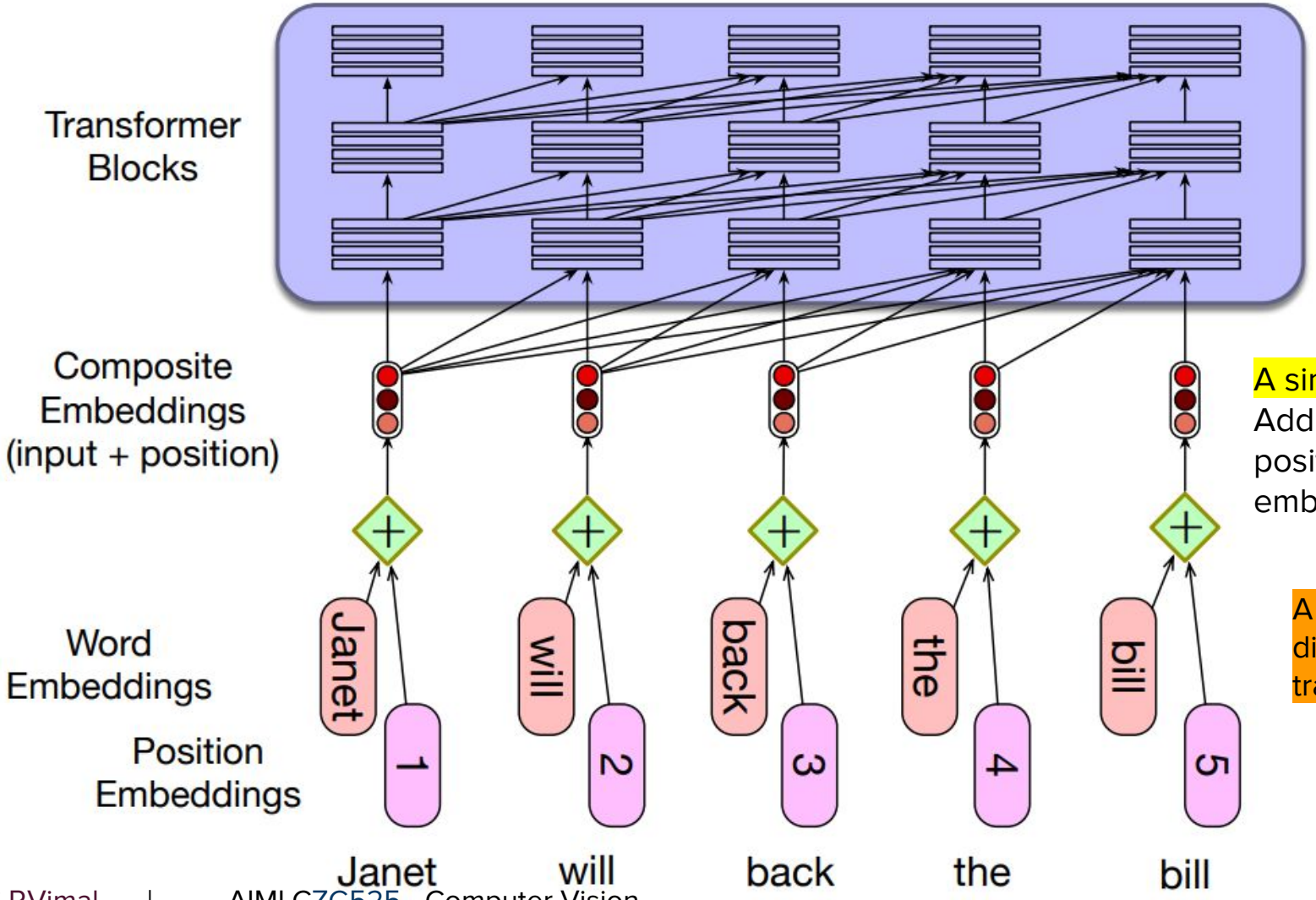
$$\mathbf{head}_i = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$$

(Review) Multihead-Attention | Transformers



Each of the multihead self-attention layers is provided with its own set of key, query and value weight matrices.
The outputs from each of the layers are concatenated and then projected down to d, thus producing an output of the same size as the input so layers can be stacked.

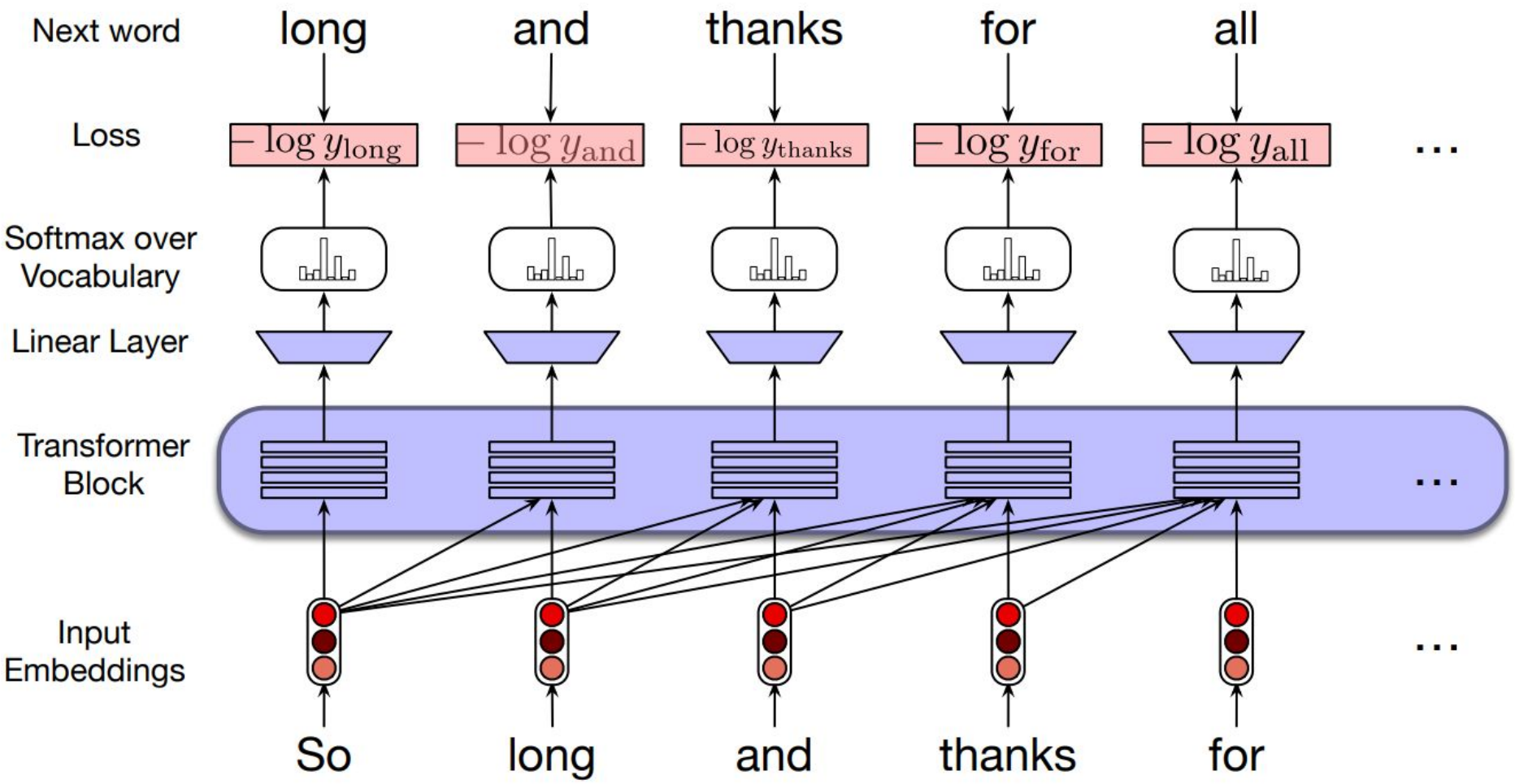
(Review) Positional Embeddings | Transformers



A simple way to model position:
Add an embedding representation of the absolute position to the input word embedding to produce a new embedding of the same dimensionality.

A combination of sine and cosine functions with differing frequencies was used in the original transformer work.

(Review) Transformers as Language Models



$$= \frac{1}{T}\sum_{t=1}^{T} L_{CE}$$

Vision Transformers

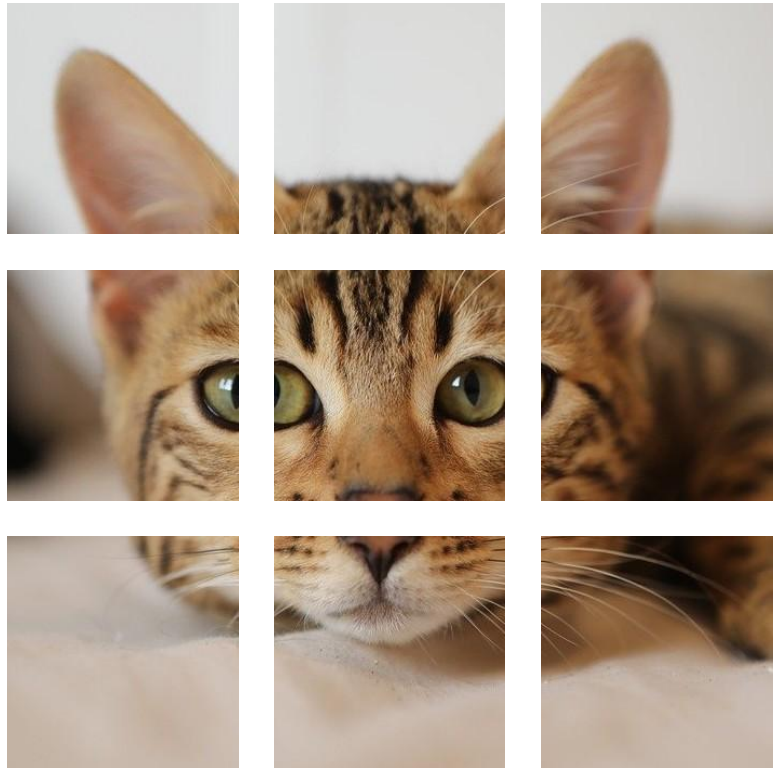
Vision Transformer (ViT)

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)



Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

N input patches, each of shape 3x16x16

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

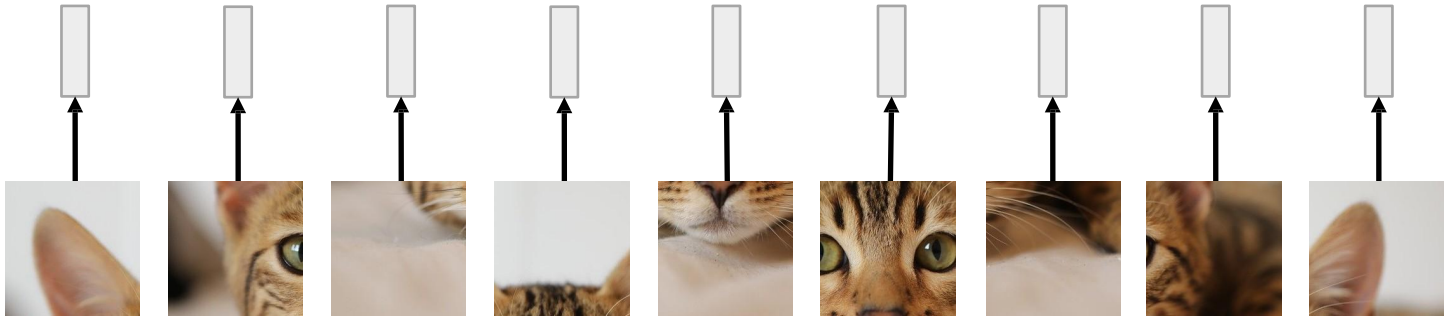
Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

Linear projection to D-dimensional vector

N input patches, each of shape 3x16x16



Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

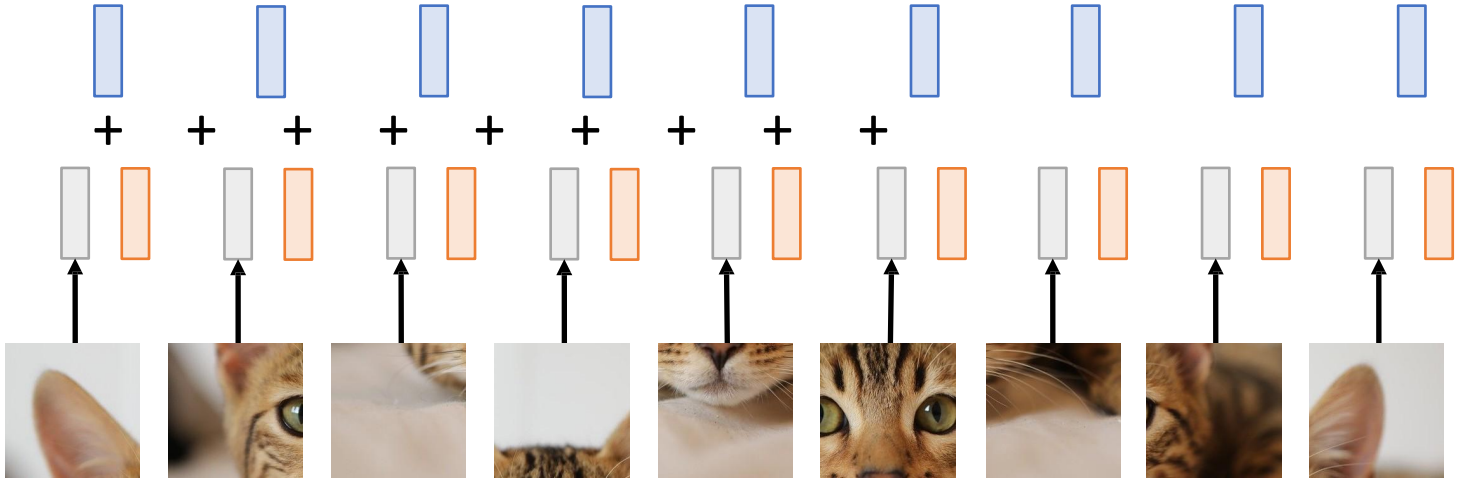
Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

Add positional embedding: learned D-dim vector per position

Linear projection to D-dimensional vector

N input patches, each of shape 3x16x16



Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

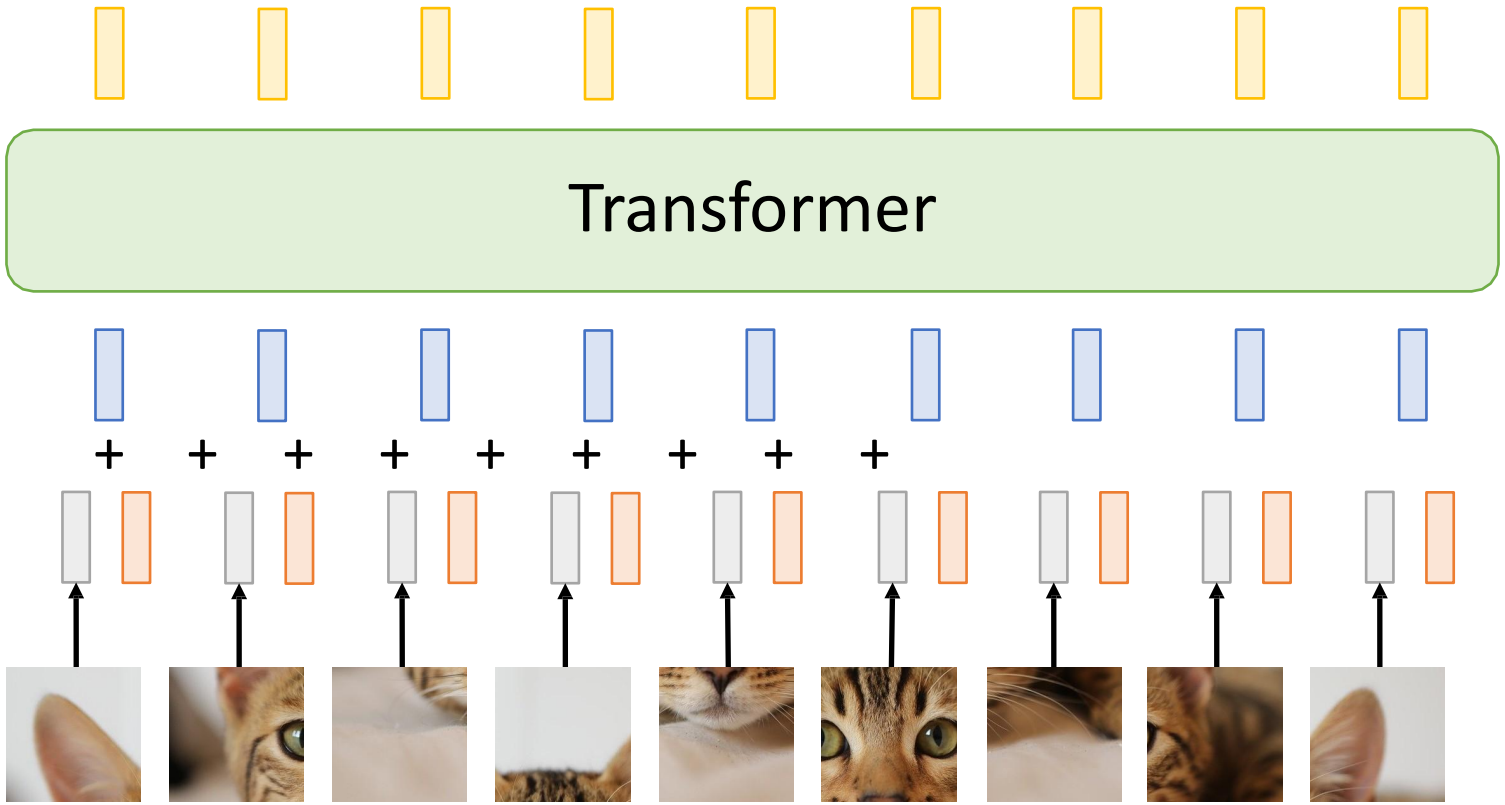
Output vectors

Exact same as NLP Transformer!

Add positional embedding: learned D-dim vector per position

Linear projection to D-dimensional vector

N input patches, each of shape 3x16x16



Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

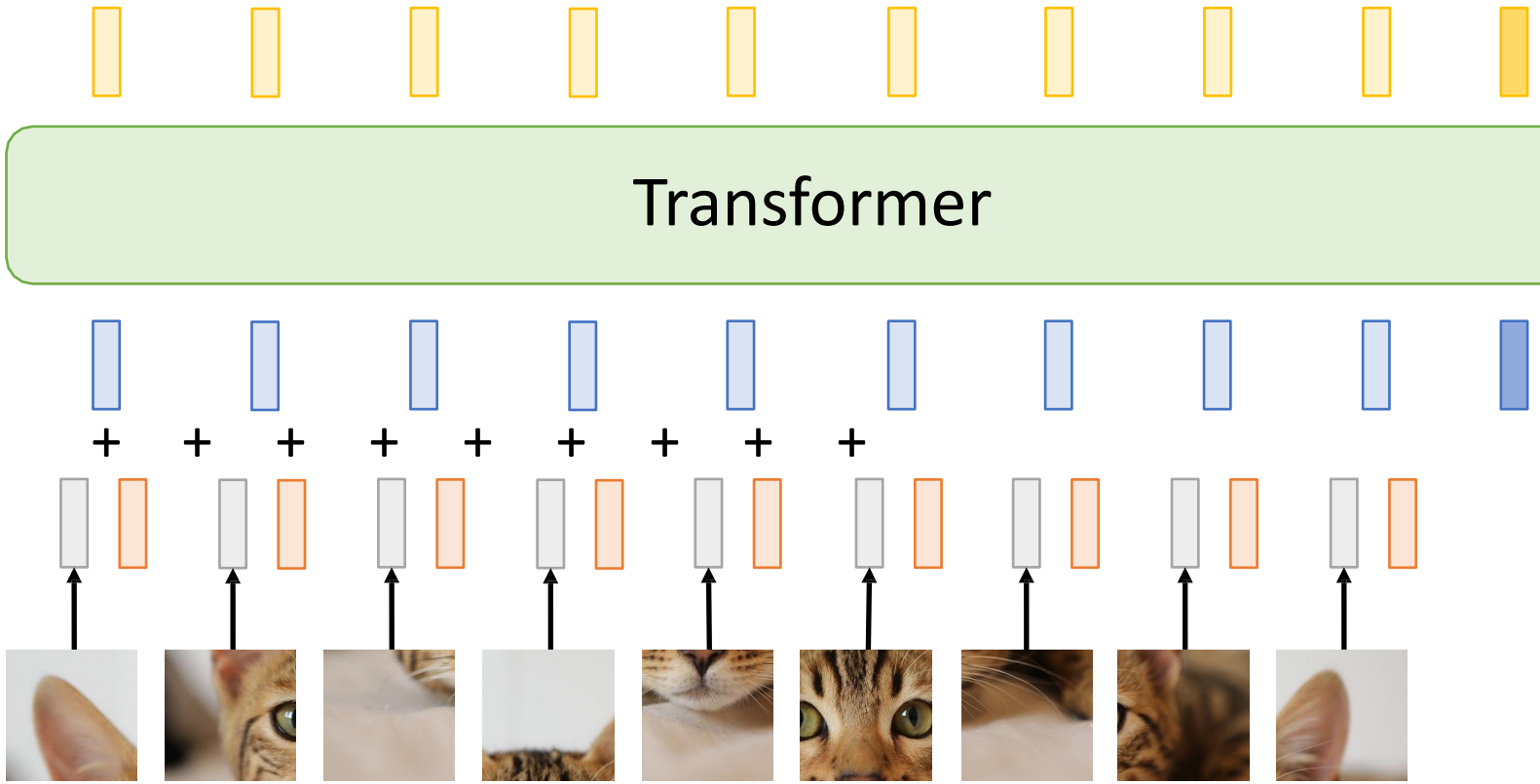
Output vectors

Exact same as NLP Transformer!

Add positional embedding: learned D-dim vector per position

Linear projection to D-dimensional vector

N input patches, each of shape 3x16x16



Special extra input: **classification token** (D dims, learned)

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

Linear projection to C-dim vector of predicted class scores

Output vectors

Exact same as NLP Transformer!

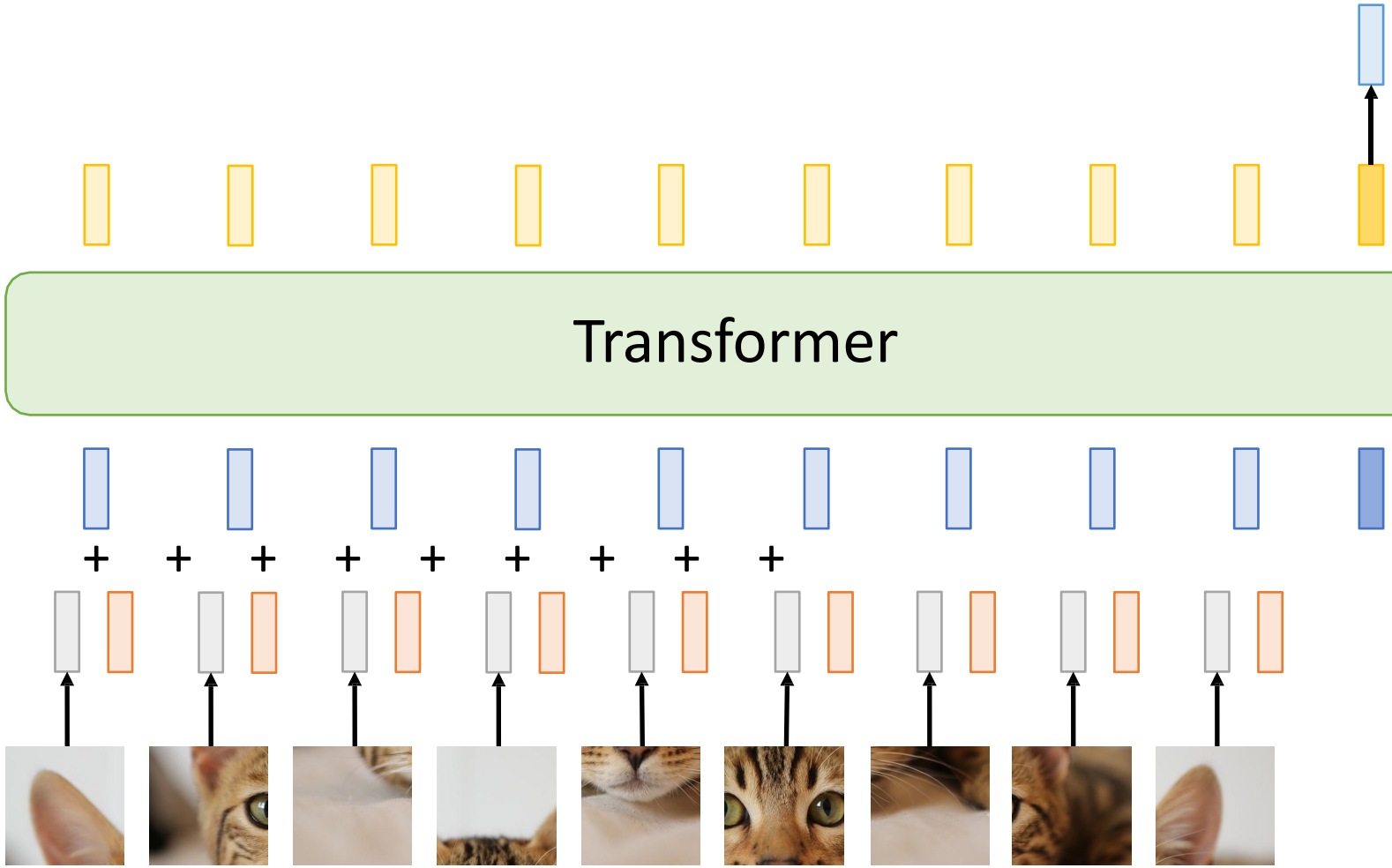
Transformer

Add positional embedding: learned D-dim vector per position

Special extra input: **classification token** (D dims, learned)

Linear projection to D-dimensional vector

N input patches, each of shape 3x16x16



Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

Computer vision model with no convolutions!

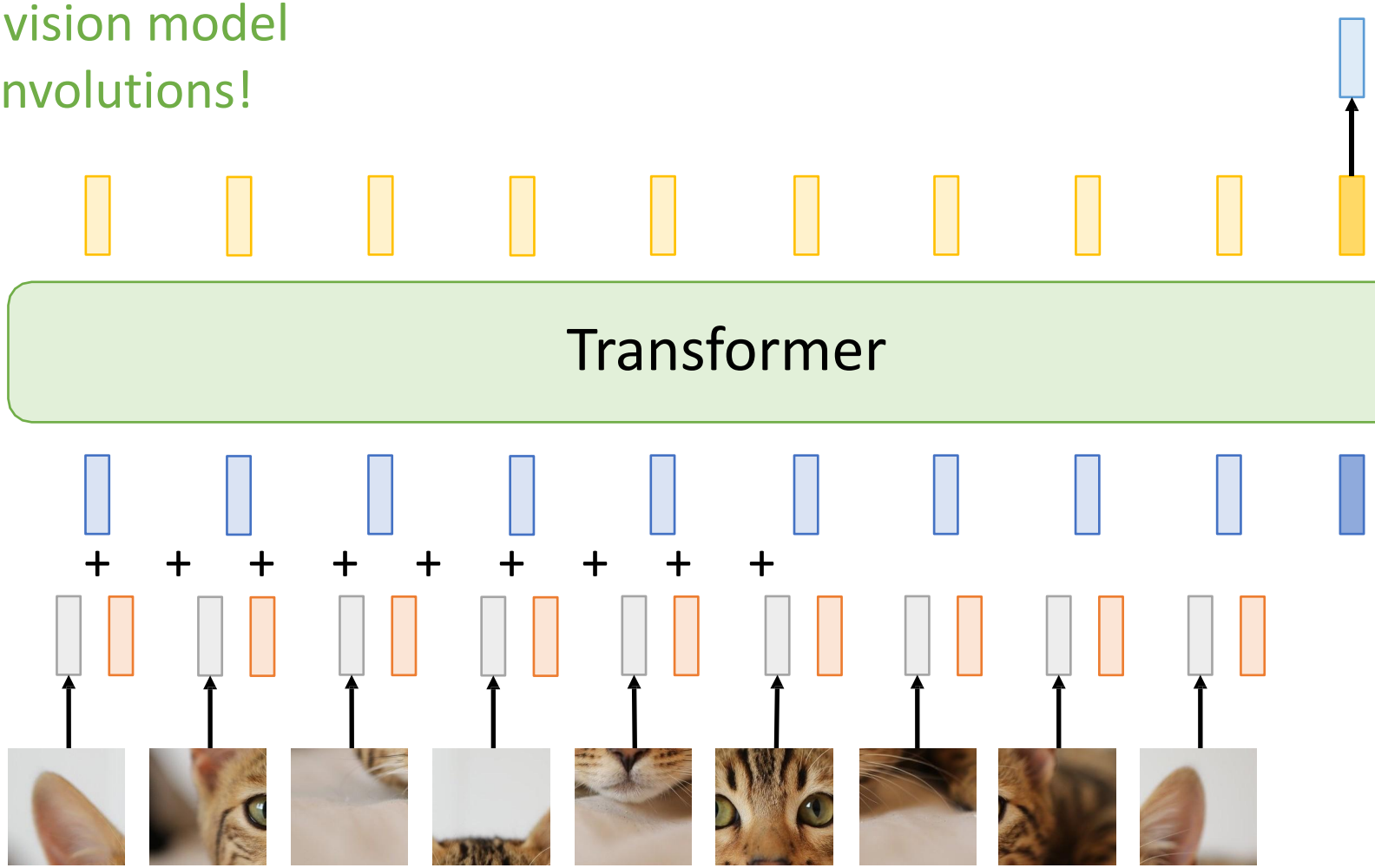
Output vectors

Exact same as NLP Transformer!

Add positional embedding: learned D-dim vector per position

Linear projection to D-dimensional vector

N input patches, each of shape 3x16x16



Linear projection to C-dim vector of predicted class scores

Special extra input: **classification token** (D dims, learned)

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

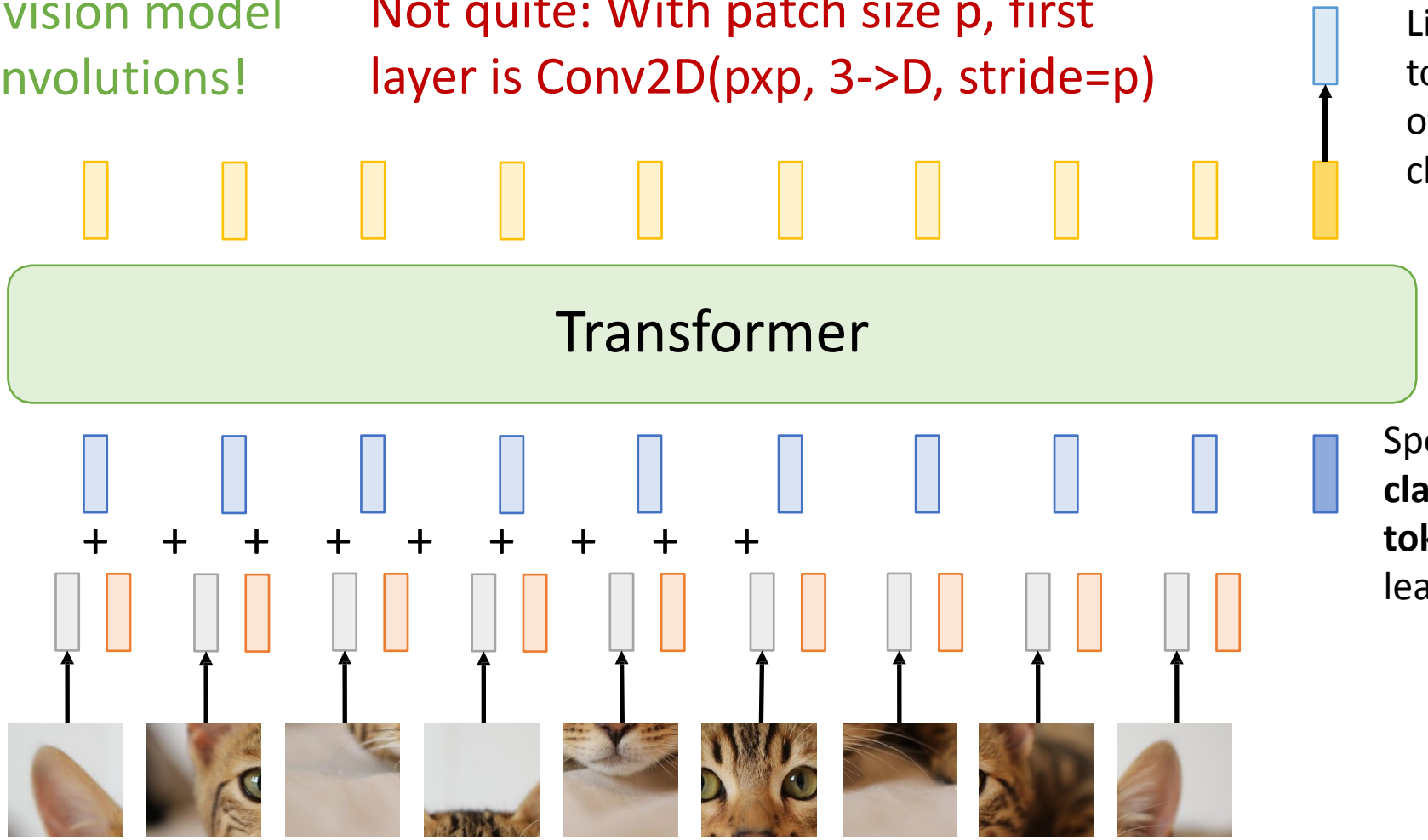
Computer vision model with no convolutions!

Not quite: With patch size p, first layer is Conv2D(pxp, 3->D, stride=p)

Linear projection to C-dim vector of predicted class scores

Output vectors

Exact same as NLP Transformer!



Add positional embedding: learned D-dim vector per position

Special extra input: **classification token** (D dims, learned)

Linear projection to D-dimensional vector

N input patches, each of shape 3x16x16

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

Computer vision model with no convolutions!

Not quite: MLPs in Transformer are stacks of 1x1 convolution

Linear projection to C-dim vector of predicted class scores

Output vectors

Exact same as NLP Transformer!

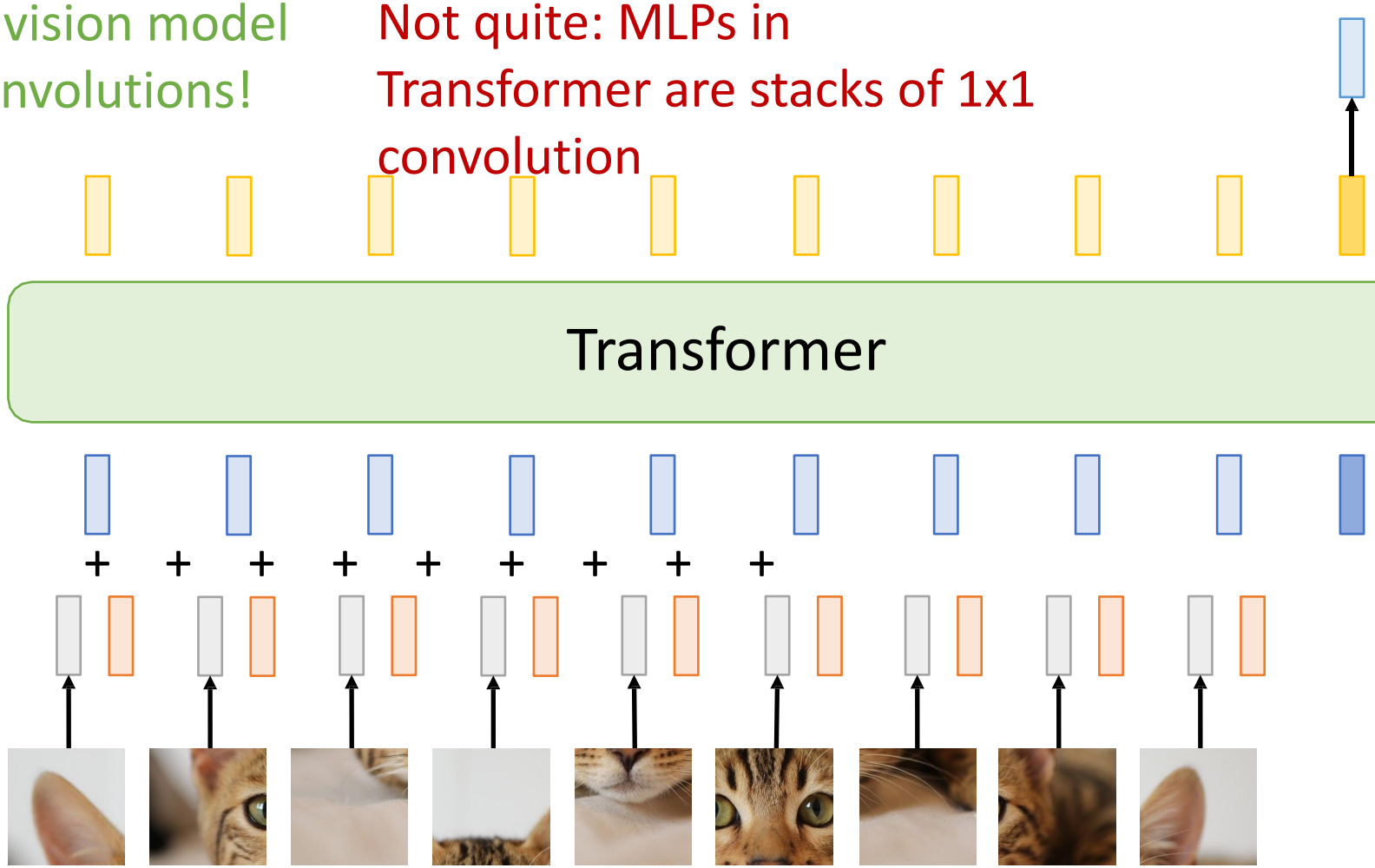
Transformer

Add positional embedding: learned D-dim vector per position

Special extra input: **classification token** (D dims, learned)

Linear projection to D-dimensional vector

N input patches, each of shape 3x16x16



Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

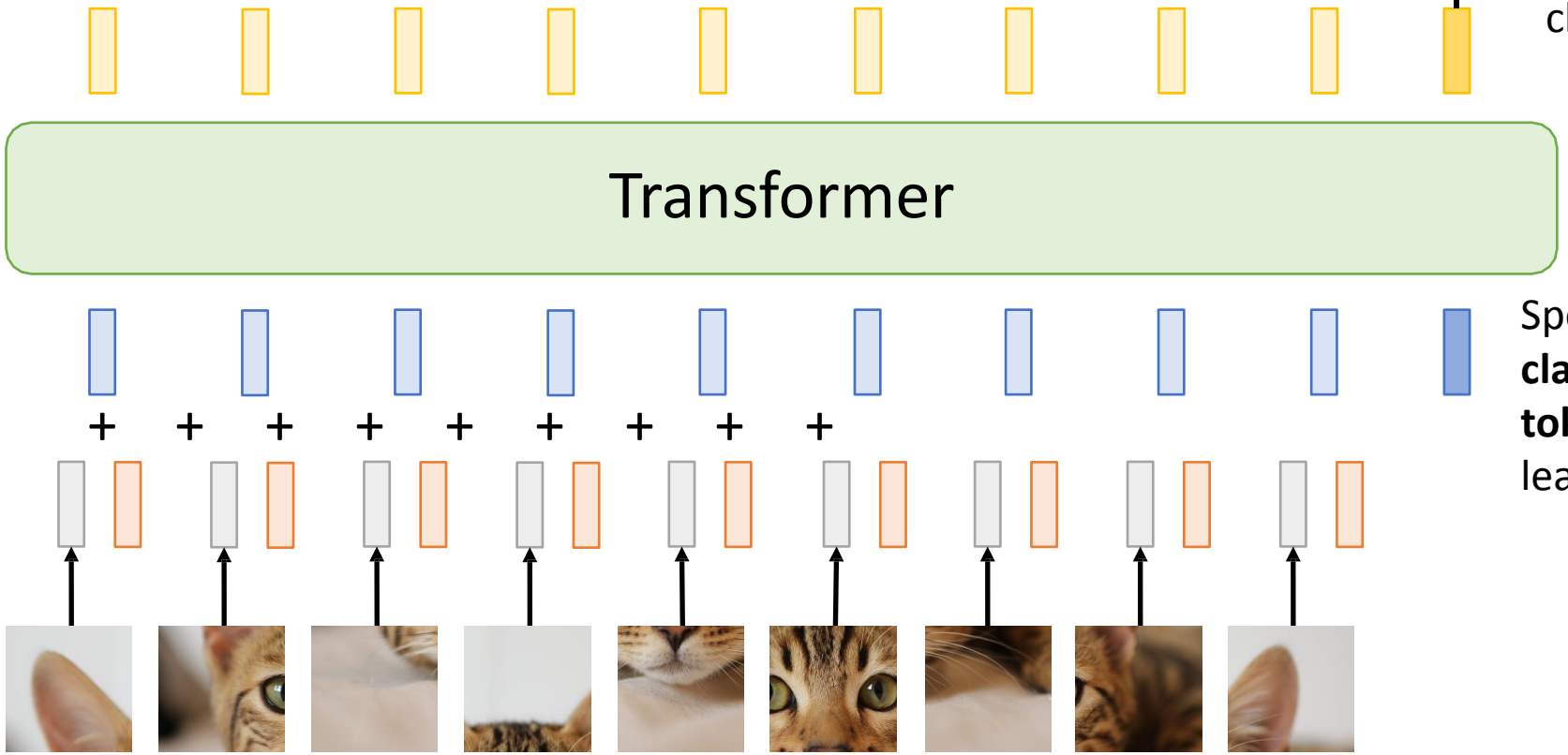
In practice: take 224x224 input image, divide into 14x14 grid of 16x16 pixel patches (or 16x16 grid of 14x14 patches)

Each attention matrix has $14^4 = 38{,}416$ entries, takes 150 KB (or 65,536 entries, takes 256 KB)

Linear projection to C-dim vector of predicted class scores

Output vectors

Exact same as NLP Transformer!



Add positional embedding: learned D-dim vector per position

Special extra input: **classification token** (D dims, learned)

Linear projection to D-dimensional vector

N input patches, each of shape 3x16x16

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

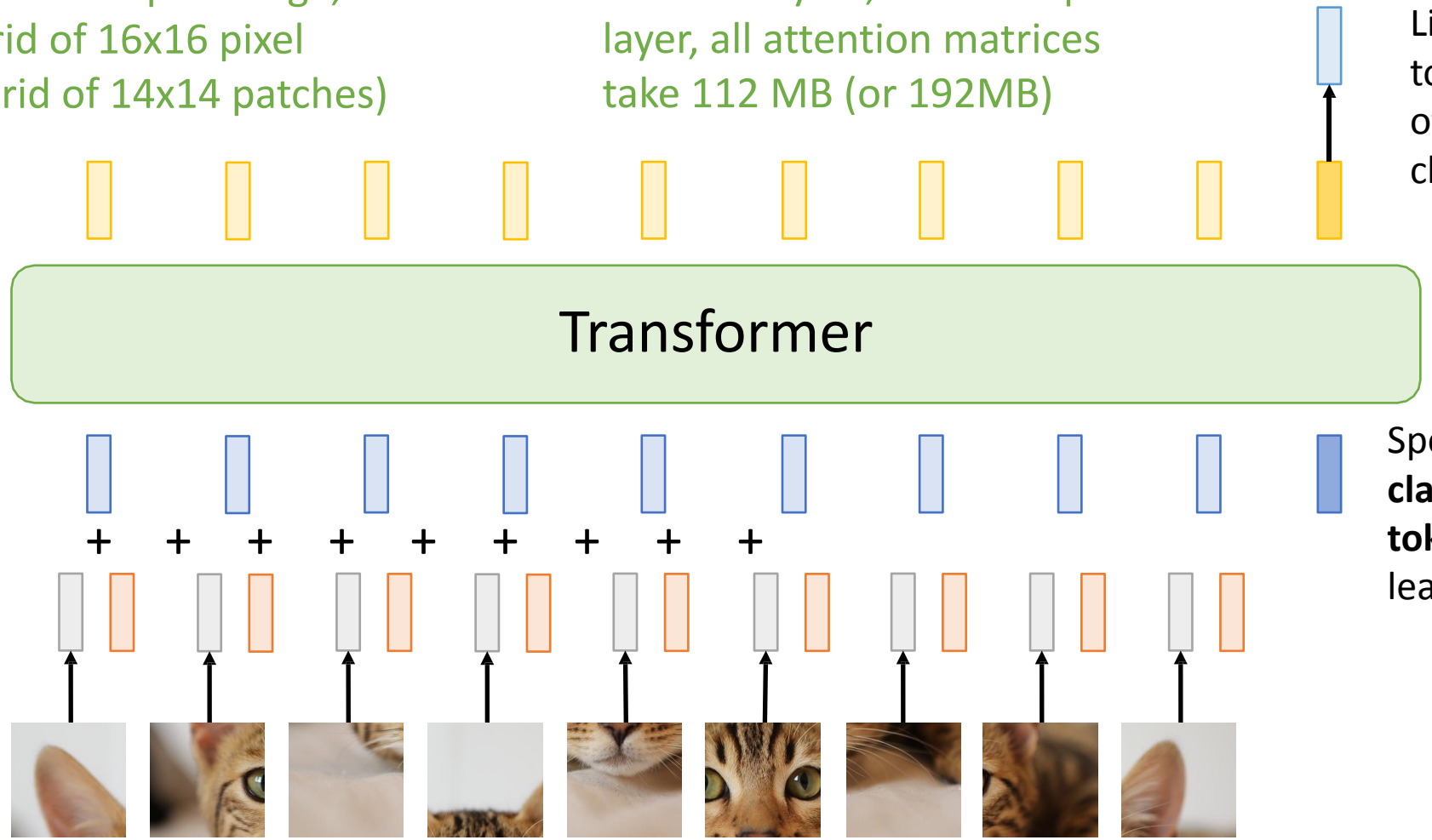
In practice: take 224x224 input image, divide into 14x14 grid of 16x16 pixel patches (or 16x16 grid of 14x14 patches)

With 48 layers, 16 heads per layer, all attention matrices take 112 MB (or 192MB)

Linear projection to C-dim vector of predicted class scores

Output vectors

Exact same as NLP Transformer!



Add positional embedding: learned D-dim vector per position

Special extra input: **classification token** (D dims, learned)

Linear projection to D-dimensional vector

N input patches, each of shape 3x16x16

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

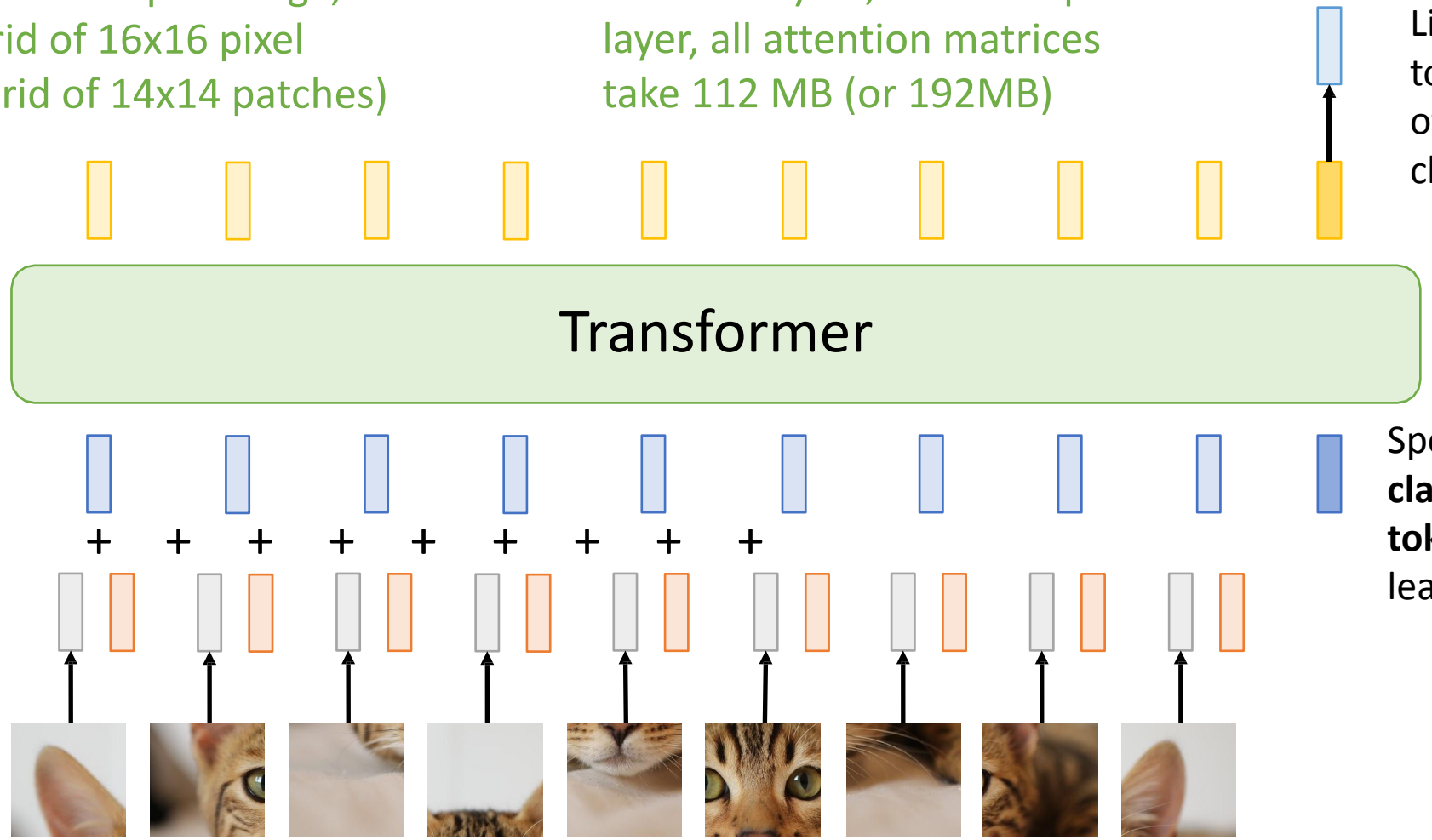
In practice: take 224x224 input image, divide into 14x14 grid of 16x16 pixel patches (or 16x16 grid of 14x14 patches)

With 48 layers, 16 heads per layer, all attention matrices take 112 MB (or 192MB)

Linear projection to C-dim vector of predicted class scores

Output vectors

Exact same as NLP Transformer!



Add positional embedding: learned D-dim vector per position

Special extra input: **classification token** (D dims, learned)

Linear projection to D-dimensional vector

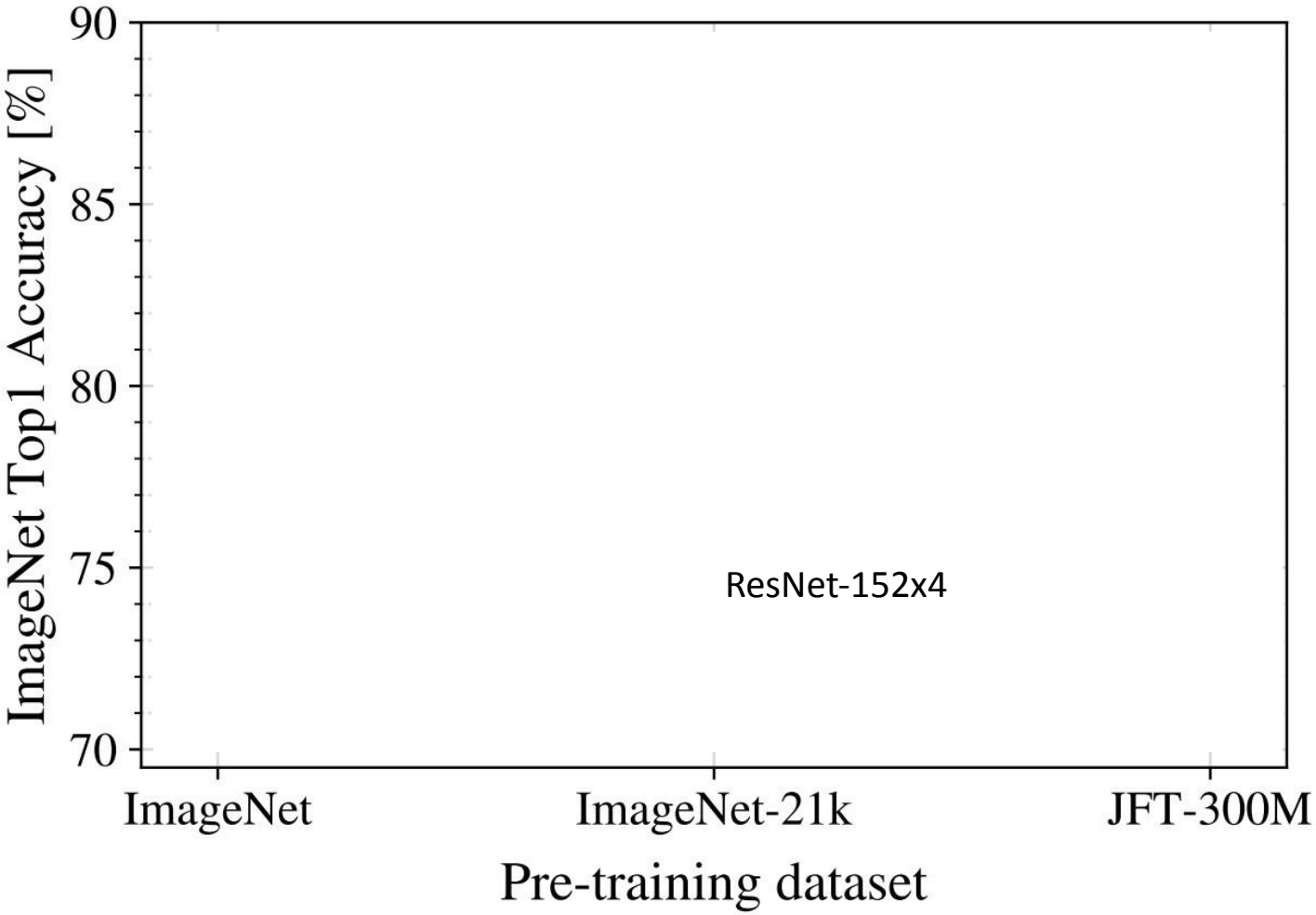
N input patches, each of shape 3x16x16

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets



B = Base
L = Large
H = Huge

/32, /16, /14 is patch size; smaller patch size is a bigger model (more patches)

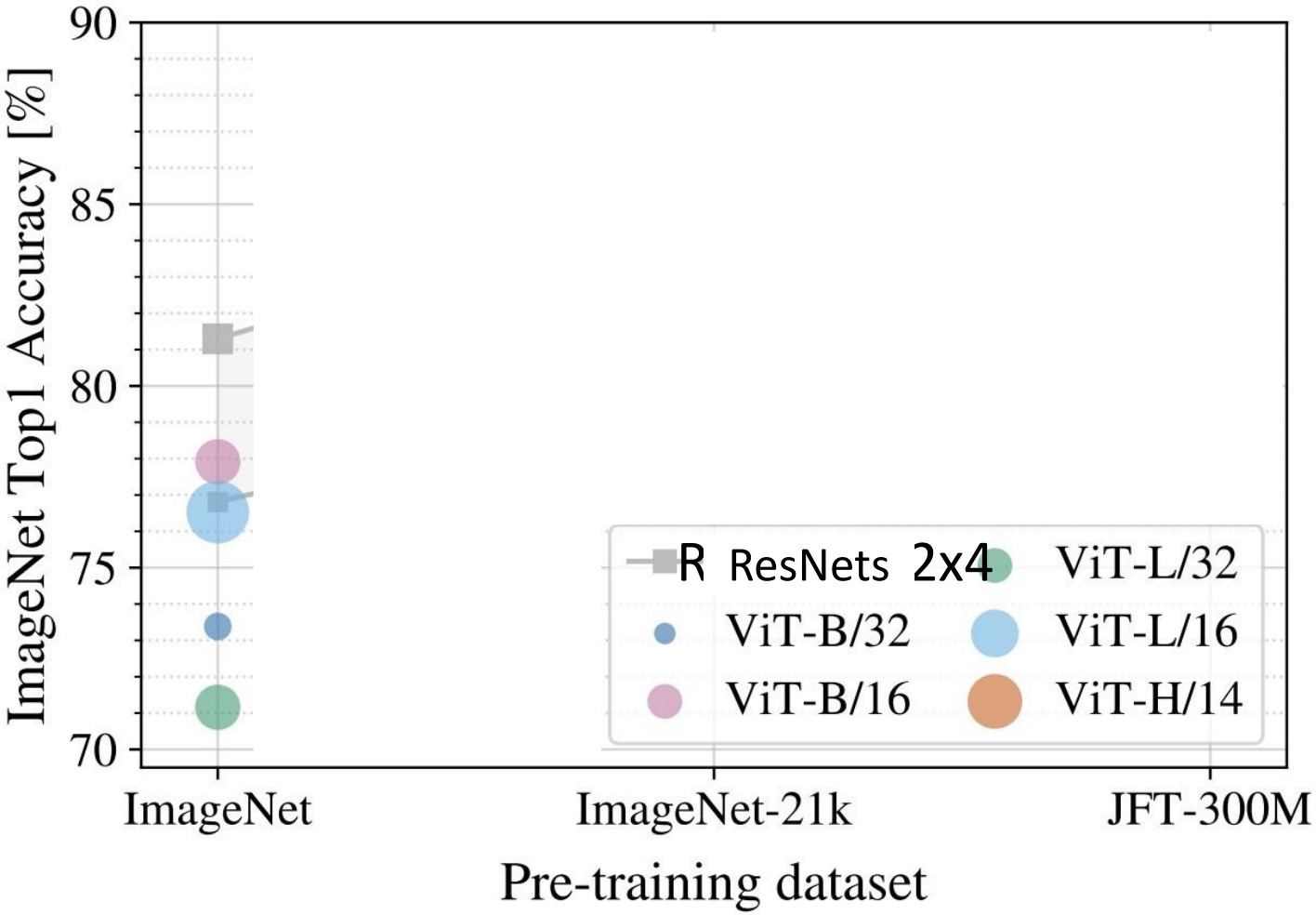
Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets

Recall: ImageNet dataset has 1k categories, 1.2M images

When trained on ImageNet, ViT models perform worse than ResNets



B = Base
L = Large
H = Huge

/32, /16, /14 is patch size; smaller patch size is a bigger model (more patches)

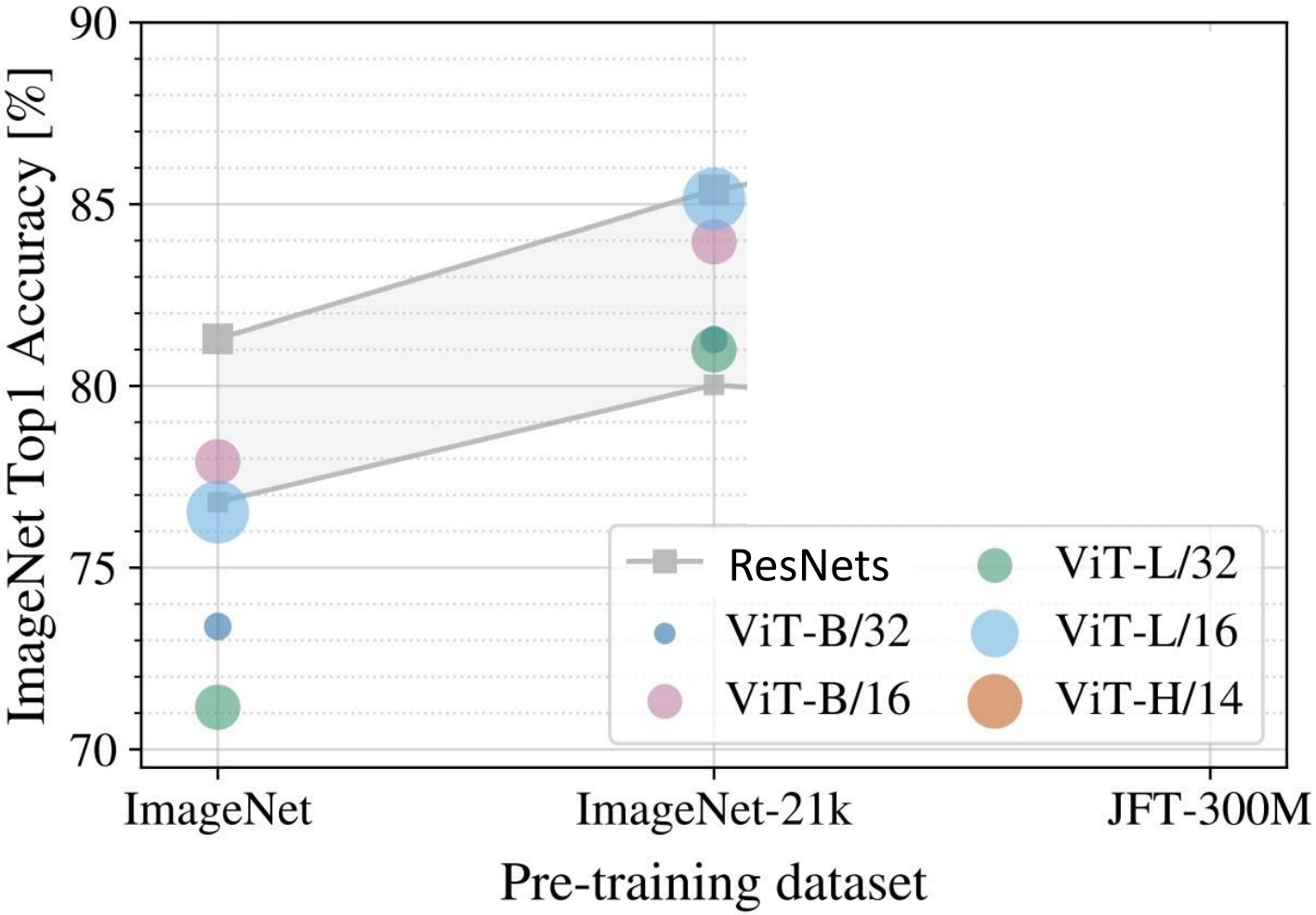
Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets

ImageNet-21k has 14M images with 21k categories

If you pretrain on ImageNet-21k and fine-tune on ImageNet, ViT does better: big ViTs match big ResNets



B = Base
L = Large
H = Huge

/32, /16, /14 is patch size; smaller patch size is a bigger model (more patches)

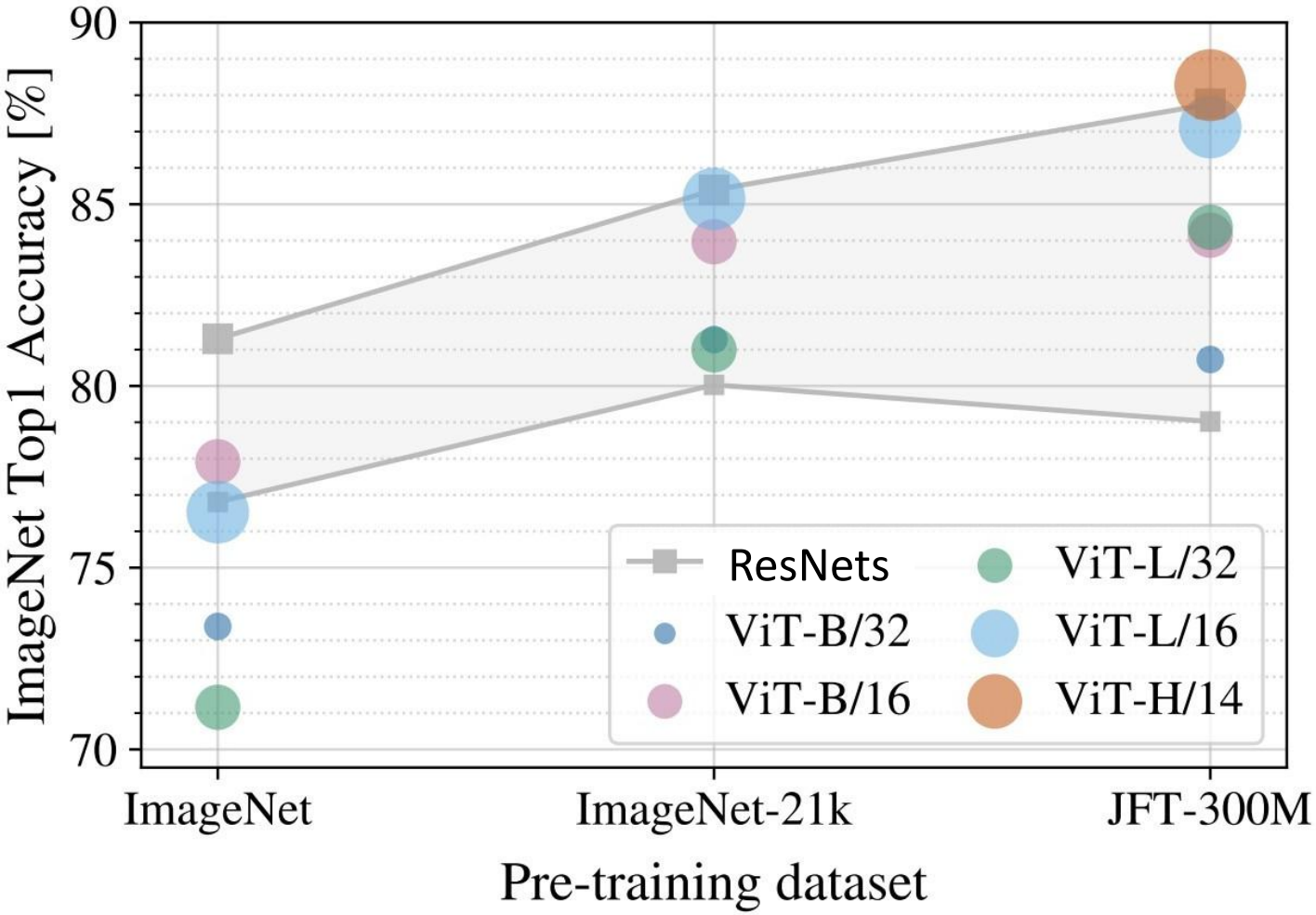
Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets

JFT-300M is an internal Google dataset with 300M labeled images

If you pretrain on JFT and finetune on ImageNet, large ViTs outperform large ResNets



B = Base
L = Large
H = Huge

/32, /16, /14 is patch size; smaller patch size is a bigger model (more patches)

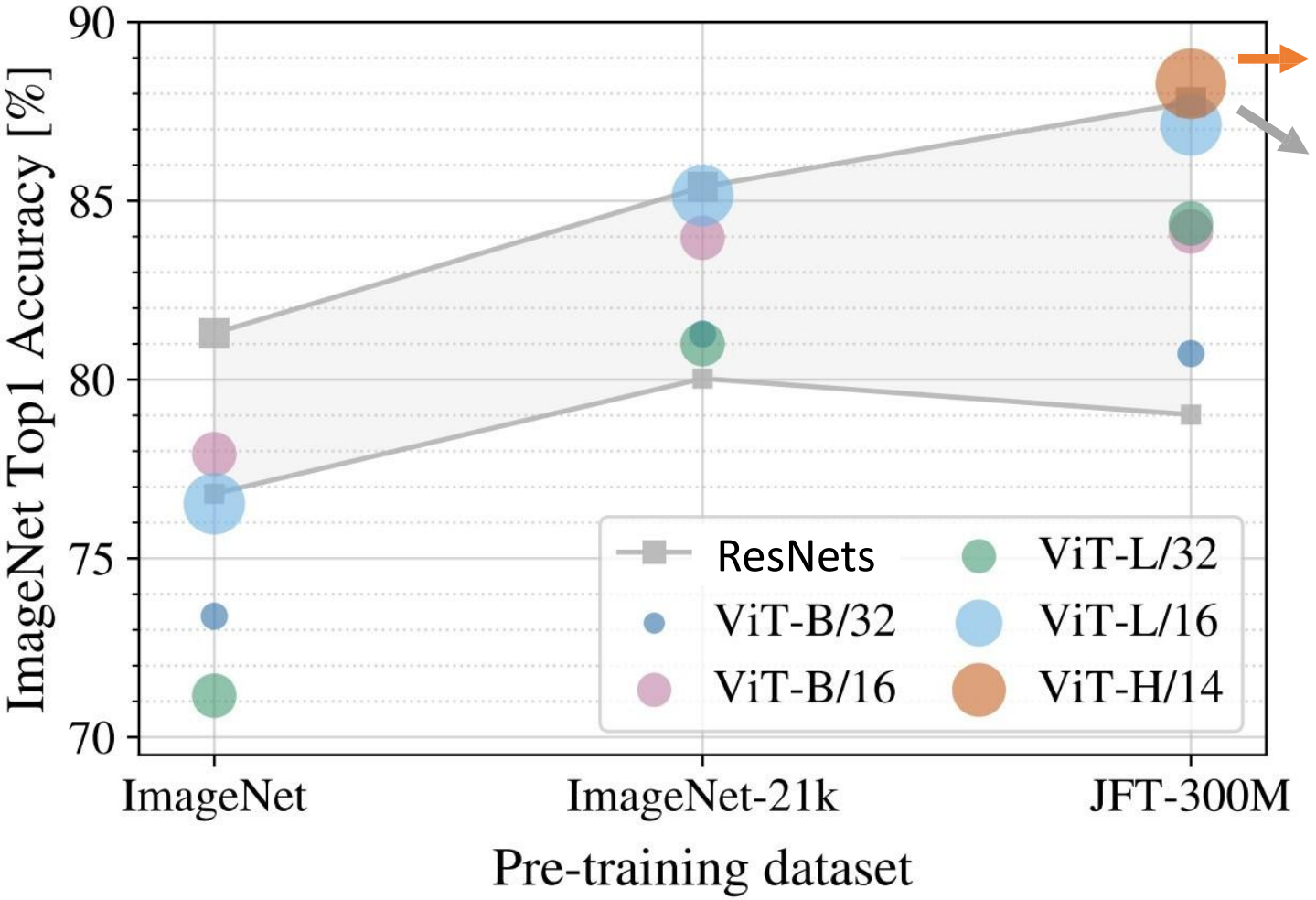
Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets

JFT-300M is an internal Google dataset with 300M labeled images

If you pretrain on JFT and finetune on ImageNet, large ViTs outperform large ResNets



ViT: 2.5k TPU-v3 core days of training

ResNet: 9.9k TPU-v3 core days of training

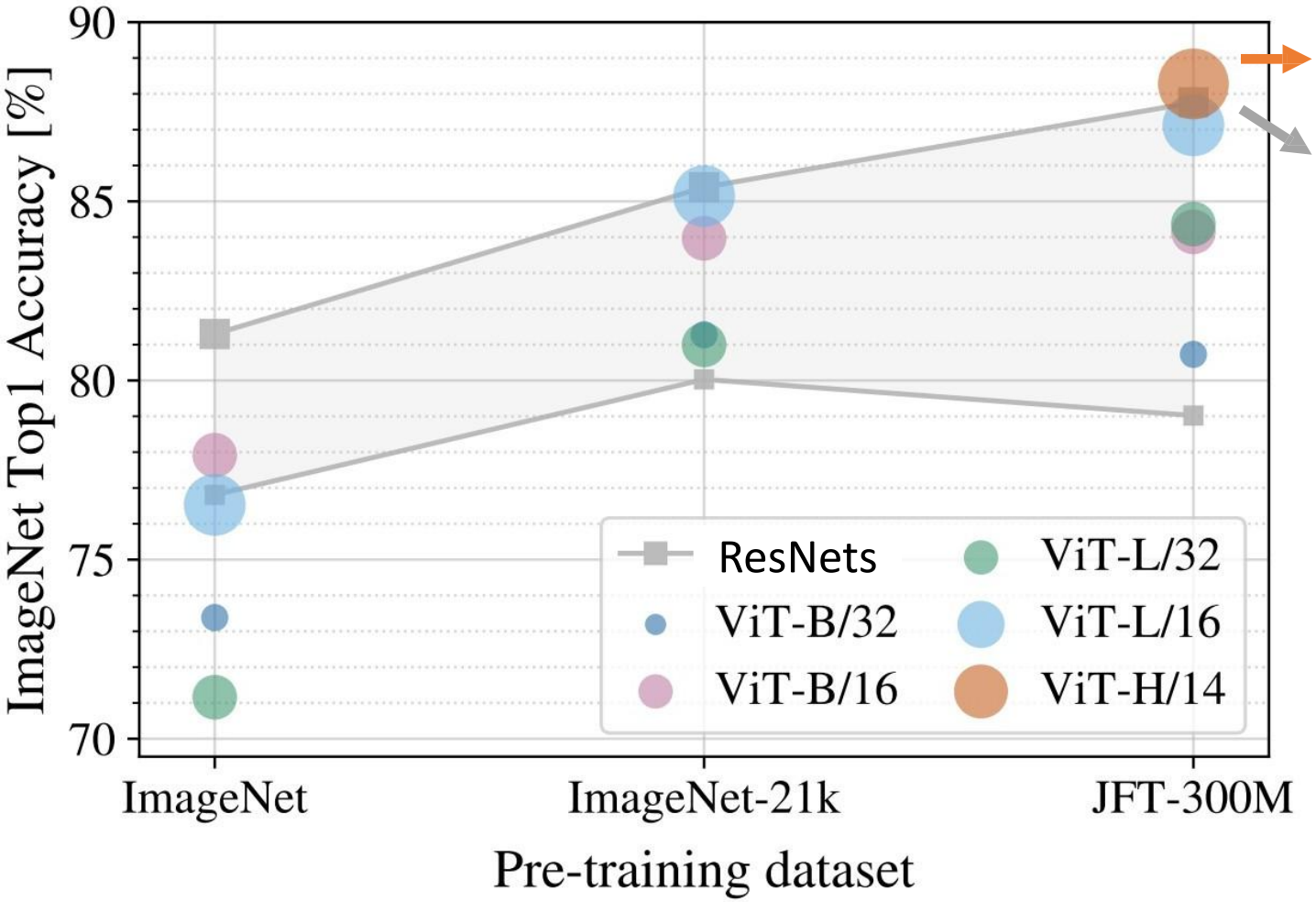
ViTs make more efficient use of GPU / TPU hardware (matrix multiply is more hardware-friendly than conv)

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets

Claim: ViT models have “less inductive bias” than ResNets, so need more pretraining data to learn good features
(Not sure I buy this explanation: “inductive bias” is not a well-defined concept we can measure!)



ViT: 2.5k TPU-v3 core days of training

ResNet: 9.9k TPU-v3 core days of training

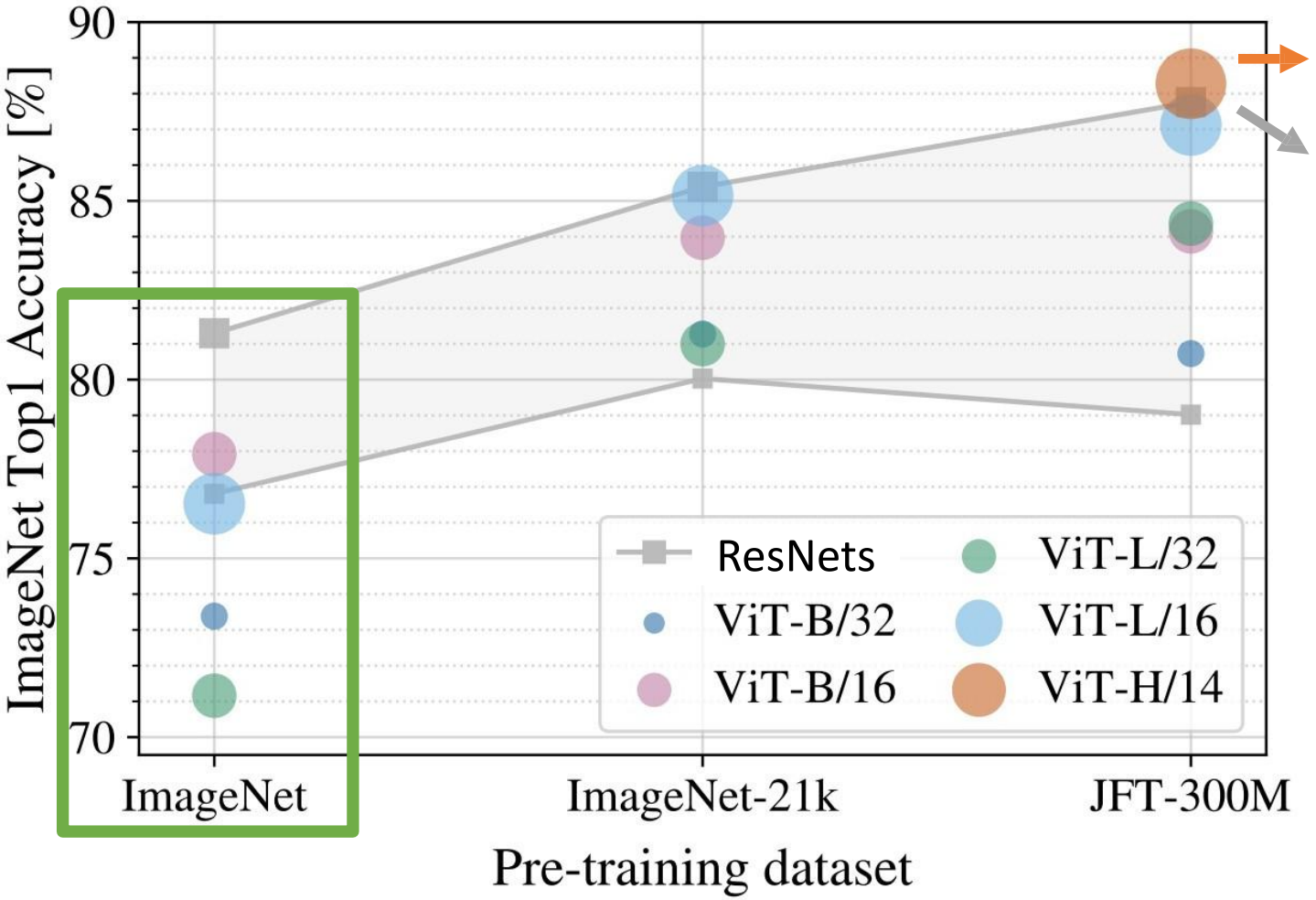
ViTs make more efficient use of GPU / TPU hardware (matrix multiply is more hardware-friendly than conv)

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets

How can we improve the performance of ViT models on ImageNet?



ViT: 2.5k TPU-v3 core days of training

ResNet: 9.9k TPU-v3 core days of training

ViTs make more efficient use of GPU / TPU hardware (matrix multiply is more hardware-friendly than conv)

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

- ***Additional Readings:***
    - [*Recommended*] An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale - Alexey Dosovitskiy, ... 2020, CVPR

BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Thank you